TU WIEN Informatics

Reducing Token Usage of Software Engineering Agents

DIPLOMARBEIT

zur Erlangung des akademischen Grades

Diplom-Ingenieur

im Rahmen des Studiums

Software Engineering

eingereicht von

Nicolas Hrubec, BSc
Matrikelnummer 11722652

an der Fakultät für Informatik

der Technischen Universität Wien

Betreuung: Associate Prof. Dipl.-Ing. Dr.sc. Jürgen Cito, BSc

Wien, 12. November 2025

Nicolas Hrubec	Jürgen Cito

TU WIEN Informatics

Reducing Token Usage of Software Engineering Agents

DIPLOMA THESIS

submitted in partial fulfillment of the requirements for the degree of

Diplom-Ingenieur

in

Software Engineering

by

Nicolas Hrubec, BSc
Registration Number 11722652

to the Faculty of Informatics

at the TU Wien

Advisor: Associate Prof. Dipl.-Ing. Dr.sc. Jürgen Cito, BSc

Vienna, November 12, 2025

Nicolas Hrubec	Jürgen Cito

Technische Universität Wien
A-1040 Wien ▪ Karlsplatz 13 ▪ Tel. +43-1-58801-0 ▪ www.tuwien.at

Erklärung zur Verfassung der Arbeit

Nicolas Hrubec, BSc

Hiermit erkläre ich, dass ich diese Arbeit selbständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit – einschließlich Tabellen, Karten und Abbildungen –, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind, auf jeden Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.
Ich erkläre weiters, dass ich mich generativer KI-Tools lediglich als Hilfsmittel bedient habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Im Anhang „Übersicht verwendeter Hilfsmittel“ habe ich alle generativen KI-Tools gelistet, die verwendet wurden, und angegeben, wo und wie sie verwendet wurden. Für Textpassagen, die ohne substantielle Änderungen übernommen wurden, haben ich jeweils die von mir formulierten Eingaben (Prompts) und die verwendete IT- Anwendung mit ihrem Produktnamen und Versionsnummer/Datum angegeben.

Wien, 12. November 2025

Nicolas Hrubec

Danksagung

Zunächst möchte ich meinen tiefsten Dank an Associate Professor Dr. Jürgen Cito aussprechen, der mir die Möglichkeit gegeben hat, diese Arbeit zu erstellen. Vielen Dank, dass du mich als Studierenden aufgenommen und mich sowohl während meiner Bachelor- als auch meiner Masterarbeit kontinuierlich begleitet hast. Besonders dankbar bin ich für das Vertrauen und die Flexibilität, die es mir ermöglicht haben, diese Arbeit nach meinen eigenen Vorstellungen zu gestalten.

Des Weiteren möchte ich meiner Familie für ihre anhaltende emotionale und finanzielle Unterstützung in all diesen Jahren danken.

Abschließend möchte ich auch meinen Freundinnen und Freunden für ihre Unterstützung danken, insbesondere Kathi, die stets ein offenes Ohr hatte, wenn es mal nicht nach meinen Vorstellungen gelaufen ist.

Acknowledgements

I would like to express my deepest gratitude to Associate Professor Dr. Jürgen Cito for granting me the opportunity to pursue this work. Thank you for taking me on as a student and for your continuous guidance throughout both my bachelor's and master's theses. I am particularly grateful for the trust and flexibility you offered, which enabled me to shape this work according to my own ideas.

I also wish to thank my family for their unwavering emotional and financial support throughout these years.

Finally, I extend my sincere thanks to my friends for their encouragement and companionship — especially Kathi, for always having an open ear and patiently listening to me when things didn't go as planned.

Kurzfassung

Auf LLMs basierende Agenten werden zunehmend eingesetzt, um Aufgaben in der Softwareentwicklung, wie die Behebung von Bugs oder die Implementierung neuer Funktionen, zu automatisieren. Solche Systeme müssen Informationen aus großen Code-Repositorien verarbeiten, wodurch ihr Arbeitskontext schnell anwächst. Ein langer Kontext ist jedoch teuer und kann die Modellleistung beeinträchtigen, da LLMs Schwierigkeiten haben, irrelevante Informationen zu ignorieren. Diese Arbeit untersucht Strategien zur Reduktion der Kontextgröße und damit des Tokenverbrauchs in Agenten für die Softwareentwicklung, bei möglichst geringem Einfluss auf die Performanz.

Eine Voranalyse zeigt, dass Code-Tokens den größten Anteil des gesamten Tokenverbrauchs im gewählten Setup ausmachen. Aus diesem Grund schlagen wir vor, eine Reihe von Code-Minifikationstransformationen anzuwenden, die nicht essenzielle lexikalische Elemente entfernen oder verkürzen, ohne die Programmsemantik zu verändern. Die vorgeschlagenen Transformationen werden in einen Agenten für die Softwareentwicklung integriert und systematisch auf der SWE-bench Verified Benchmark, unter Verwendung von GPT-4.1 und GPT-5-mini, evaluiert.

Die Experimente zeigen, dass Minifikation den durchschnittlichen Eingabe-Tokenverbrauch um 42% reduziert, bei einem Leistungsverlust von lediglich 12%. Diese Ergebnisse verdeutlichen, dass einfache Code-Transformationen erhebliche Effizienzgewinne bei gleichzeitig hoher Leistungsfähigkeit ermöglichen und somit einen vielversprechenden Ansatz für kosteneffizientere Agenten darstellen.

Abstract

LLM-based agents are increasingly employed to automate software engineering tasks such as bug fixing and feature implementation. These systems must reason over large code repositories, causing their working context to grow rapidly. Large contexts are costly to process and can degrade model performance, as LLMs struggle to disregard irrelevant information. This thesis investigates strategies to reduce context size and therefore token usage in SWE agents, while minimizing performance impact.

Preliminary analysis reveals that source code tokens dominate overall token consumption in the chosen setup, motivating a focus on code-level transformations. We therefore propose to apply a series of code minification techniques that remove or shorten non-essential lexical elements while preserving program semantics. The proposed transformations are integrated into a state-in-context SWE agent and systematically evaluated on the SWE-bench Verified benchmark using GPT-4.1 and GPT-5-mini.

Experiments show that minification reduces average input token usage by 42% with only a 12% drop in resolution rate. These findings demonstrate that lightweight source code transformations can yield substantial efficiency gains while maintaining strong performance, indicating a promising path towards more cost-effective agents.

Contents

CHAPTER 1

Introduction

1.1 Problem Statement and Motivation

Large language models (LLMs) are increasingly deployed as the central reasoning component in agentic systems for software engineering (SE). On repository-level tasks (e.g., SWE-bench [1]), agents (repeatedly) retrieve, read and reason over large codebases. In such workflows, context size grows quickly, directly translating to high token usage. Since token usage typically determines LLM API cost, long contexts incur high operational cost. Additionally, studies have demonstrated that LLMs can struggle to fully utilize long contexts, leading to performance degradation [2,3]. For these reasons, effectively reducing context size (while retaining relevant information) is essential for both performance and cost efficiency in LLM agents.

This thesis is concerned with reducing end-to-end token consumption in SE agents without substantially reducing task success. Concretely, we are concerned with the following guiding research question:
What is the impact of token-reducing input transformation strategies on the performance of software engineering agents?

Ideally we would like to find transformations f such that $t(f(s)) < t(s) \quad \text{and} \quad p(f(s)) \geq p(s) - \varepsilon$.

where:
- s : input states (prompts before transformation)
- $t(s)$: mean token cost of state s
- $p(s)$: mean task success probability of state s
- ε : maximum acceptable performance degradation

With the growing use of LLM agents in SE, even marginal improvements in token efficiency can produce substantial aggregate energy and cost savings. By demonstrating token reductions while maintaining adequate performance, this work aims to improve the operational scalability and energy efficiency of LLM-based software-engineering agents.

1.2 Research Questions

The main goal of this thesis is to design, implement and evaluate input transformations that reduce context size and therefore token consumption in LLM-based SE agents. To make progress towards this goal, the following sub-questions will be explored:

RQ1: How are tokens allocated in LLM-based software-engineering agents? In this question, we empirically investigate token allocation across the end-to-end workflow of SE agents. We measure the following contributions to the overall token budget:

- What proportion of tokens corresponds to code versus natural language?
- What proportion of tokens correspond to retrieval versus program repair?
- How are tokens distributed across syntactic and lexical units (e.g., signature, identifiers, comments/docstrings, whitespace)?

RQ2: Which token-reducing input transformation strategies are suitable to effectively address observed token bottlenecks? The preliminary analysis will uncover token bottlenecks and guide the choice and design of candidate transformation strategies.

RQ3: What is the impact of implemented input transformation strategies on token consumption and task performance? We empirically evaluate the impact of selected transformations on token consumption and performance on SE benchmarks.

RQ4: How robust and generalizable are observed savings and trade-offs? We will conduct additional analyses on the results obtained from RQ3. Specifically, we will examine whether the observed patterns remain consistent across different model choices, repositories and levels of problem difficulty. Furthermore, we will perform a detailed failure analysis to identify and categorize the main sources of model errors introduced by the implemented transformations.

1.3 Solution Approach

Preliminary analysis shows that source code tokens account for the majority of context in the studied SE agent setup. To address this imbalance, the proposed solution focuses on reducing the code portion of the prompt rather than optimizing natural language

components. Specifically, we employ transformations based on code minification. These transformations remove or alter non-essential lexical and syntactic elements such as whitespace, comments and identifier names while preserving program semantics. By applying minification before the code is passed to the agent, we aim to reduce token consumption without degrading the model's ability to reason about the underlying logic.

1.4 Structure

The remainder of this thesis is structured as follows.

Chapter 2 introduces the foundations of LLM-based agents with a focus on software engineering. We review benchmarks for evaluating SE agents and outline context-related challenges in LLMs. We also provide basic background on LLM tokenization and minification.

Chapter 3 surveys prior research on managing long contexts in LLMs. We discuss prompt compression methods, including filtering, summarization and distillation techniques. We also cover retrieval-augmented generation (RAG) and context isolation. Finally, we present research proposed for context management in SE agents.

Chapter 4 describes the implementation. We outline repository collection and preliminary analysis. We also detail the agent setup used in our experiments. Furthermore, we introduce the proposed token reduction transformations and how they are integrated into the agent.

Chapter 5 presents the experimental setup, including the selected benchmark, LLM configuration, preprocessing steps and evaluation metrics. We continue with a discussion of the results, including ablation studies and a failure analysis.

Chapter 6 concludes this thesis by summarizing key insights, revisiting the research questions and discussing limitations as well as potential future work.

CHAPTER 2

Background

In this section, we introduce the conceptual groundwork for this thesis. We begin by outlining what LLM agents are and examining their architectural building blocks. We then narrow the focus to agents developed for software-engineering and survey benchmarks used to quantify the effectiveness of such agents in real-world repositories. We discuss LLM tokenization and issues arising from long context lengths in LLMs. Finally, we provide a brief introduction to source code minification.

2.1 LLM Agents

2.1.1 Overview

An autonomous agent is generally defined as "a system situated within and a part of an environment that senses that environment and acts on it, over time, in pursuit of its own agenda" [4]. Traditionally, agents often relied on hand-crafted rules or policies learned via RL [5–8]. However, such agents struggle to generalize beyond the usually narrow environment they were trained on. This differs from what we observe in humans who can learn from a wide variety of experiences and adapt to new tasks readily, even if given only limited data [9].

In recent years, language models have made remarkable progress, demonstrating human-level intelligence across a range of domains. These capabilities are driven by large-scale training data and model sizes on the order of up to hundreds of billions of parameters [10–14]. Such models can perform reasoning and planning in ways that often appear human-like. Additionally, LLMs possess broad internal world knowledge, reducing the need for domain-specific training. Therefore, leveraging these models as the core decision-making entity for autonomous agents has emerged as a promising path towards more general agents [15–17].

2.1.2 Architecture

The architectural components of LLM-based agents can typically be classified into the categories of profiling, memory, planning and action [18], though specific agents may implement only a subset of them.

Profiling Module

Defines the agent's role or persona in the task (e.g., a software engineer). This information is commonly provided directly in the prompt to influence the behavior of the LLM. For example, Qian et al. [19] introduce a multi-agent framework called ChatDev in which distinct social roles are assigned through role-specific system prompts.

Memory Module

This module allows the agent to retain information from past interactions or world observations. There are two types of memory: short-term and long-term. In practice, LLM agents usually either rely on short-term memory only or a combination of the two. Short-term memory is realized by providing information directly in the prompt, whereas long-term memory allows for permanent storage of important observations or insights. Long-term memory can be implemented using an external (vector) database [18].

For example, Park et al. introduce Generative Agents [20], whose short-term memory holds only the immediate situational or conversational context, whereas an append-only natural-language log serves as long-term memory. Relevant fragments are retrieved from this log and periodically distilled into higher-level reflections.

Planning Module

This module is responsible for breaking down complex goals into manageable steps and deciding on a sequence of actions. Some agents use single-path planning [16, 17, 21, 22], where the agent linearly generates a step-by-step solution plan. Others employ multi-path reasoning [23] or search-based planning [24, 25], which allows the model to explore multiple possible branches of reasoning before committing to a final answer. For example, in Tree-of-Thoughts [24] the LLM can propose several alternative intermediate steps, evaluate outcomes and backtrack if needed. This multi-path planning has been shown to improve problem-solving on tasks that benefit from exploration and self-evaluation. In addition, planning can be augmented with external solvers [26, 27]. An agent might convert a high-level task into a formal planning problem and call a classical planner to find an optimal sequence of actions. This approach allows the model to leverage established algorithms for problems where LLM reasoning might be insufficient.

Action Module

The action module translates the agent's intent into concrete operations to interface with the environment. For instance, many LLM agents operate in thought-act loops, inspired

by ReAct [16]. In this setup, the agent alternates between reasoning and performing an action. This pattern allows the agent to continuously adapt its plan based on what happens after each action, instead of being forced to stick to one initial plan.

In this context, the action space defines the set of all operations available to the agent. According to Wang et al. [18], actions can be broadly divided into two categories: (1) internal actions, relying solely on the model's inherent reasoning and text generation capabilities and (2) external tool actions, which extend the agent's capabilities beyond language reasoning. Examples include code execution, web search or API access.

2.2 SWE LLM Agents

An important use-case for LLM-based agents is software engineering. In this domain the agent's task might be to fix a bug or implement a new feature in a given codebase. To achieve this, the agent must first understand existing code, locate relevant files and finally come up with patches that solve the problem while avoiding regressions. As LLM-based SWE agents are integral to this thesis, we survey some recent representative approaches below.

2.2.1 SWE-Agent

Overview

SWE-Agent [28] introduces an explicit Agent–Computer Interface (ACI) that reorganizes the software environment to optimize interaction with large language models during software-engineering tasks. The key idea is to abstract away low-level shell commands and present a set of high-level actions that an LLM can easily use to interact with the codebase. For instance, instead of relying on the LLM to come up with `grep` or `find` expressions to carry out code localization, the ACI provides simple commands like `search_file` and `search_dir`. The agent then runs in a thought-action loop following ReAct [16].

Tools

SWE-Agent's available tools include a file viewer, search tools (to search files and text within files), file editing (replace lines in files, create new files) and a `submit` command (to submit the final patch). Tool outputs are designed to avoid overwhelming the LLM. For instance, search results are limited to 50 entries. Syntax and linting errors introduced by edits are returned to the agent as feedback to facilitate debugging.

Behavior

An analysis of the agent execution traces yields distinct patterns in agent behavior. The first steps are spent on reproduction or localization of the bug. Once relevant files have been identified, the agent switches to making code edits and running the updated

code to test whether the issue has been resolved by the introduced fix. This loop can occasionally be interrupted by additional localization operations, as the agent obtains more information about the issue. Furthermore, the authors observe that the probability of resolving an issue decreases with more iterations, i.e., early submitted patches are much more likely to succeed.

Results

SWE-Agent with GPT-4 [12] achieves 12.47% on SWE-bench [1], 18.00% on SWE-bench Lite and 22.40% on SWE-bench Verified [29], with an average cost per issue of roughly 1.60 USD.

2.2.2 AutoCodeRover

Overview

AutoCodeRover [30] aims to solve issues faster and cheaper than SWE-Agent [28] by reducing the amount of irrelevant code the agent has to consider. Instead of treating the repository as a flat collection of files, AutoCodeRover works with an abstract syntax tree[1] (AST) representation of the codebase. The agent is given specialized tools to query this representation.

Workflow

The system follows a two-stage procedure. The first stage is context retrieval. During context retrieval, an agent iteratively invokes specialised search APIs to locate relevant classes, methods and code snippets. After each API call the agent evaluates whether the gathered context is sufficient, invoking additional queries if that is not the case. Each API call returns only necessary information to avoid distracting context as much as possible. For instance, during search an API call might only return the function signatures of a class instead of the full file source. The context retrieval stage results in a set of buggy locations, which are then passed to a dedicated patch-generation agent (second stage), which synthesizes a patch and applies it. In case the patch fails with syntactic or linting errors, the agent retries until a valid fix has been produced.

Results

AutoCodeRover with GPT-4 costs an average of 0.43 USD per issue while achieving a slightly higher resolve rate of 19.00% on SWE-bench Lite than SWE-Agent. At the time of writing, no results have been published for AutoCodeRover with GPT-4 on the full SWE-bench or SWE-bench Verified benchmarks.

[1] `https://docs.python.org/3/library/ast.html`

2.2.3 Agentless

Overview

While prior systems rely on complex multi-step agents with planning and tool-use capabilities, Agentless [31] questions whether such complexity is necessary. The authors demonstrate that much of the agentic overhead can be removed without degrading performance.

Workflow

The Agentless framework executes a simple three-stage pipeline:

1. **Localization**: Receives the issue description and codebase as input and transforms the codebase into a tree-like structure. The LLM is then prompted to list the N most suspicious files. Furthermore, the authors apply an embedding-based approach to retrieve additional suspicious files. After retrieving suspicious files, the LLM is provided with a skeletal view of the suspicious files (method and class signatures only) and asked to return functions and classes that should be examined more closely.

2. **Repair**: The second step in the pipeline receives the issue description and edit locations (output from the first step). Based on this information, multiple candidate patches are sampled from the LLM.

3. **Patch Validation**: To select the final patch from the candidates, the LLM first generates reproduction tests and selects a regression test suite from the existing repository tests. Next, all candidate patches are run against the selected regression suite and the patches with the lowest regression failure count are kept. Finally, the reproduction tests are run against all patches to then filter out all patches that do not successfully resolve the issue.

Results

Despite its simplicity, using GPT-4o this minimal workflow achieves a 32% resolve rate on SWE-bench Lite at an average cost of 0.70 USD per bug, significantly outperforming more complex setups like SWE-Agent and AutoCodeRover.

2.2.4 State-in-Context Agents

Overview

Most LLM-based software agents operate in partially observable environments. Given a task within a repository, the agent performs a sequence of actions to gradually collect information. The accumulated observations form a partial reconstruction of the

underlying repository, ideally containing enough information to complete the task. Existing agent frameworks implement this paradigm either through specialized tools (e.g., SWE-Agent [28]) or task-specific execution pipelines (e.g., Agentless [31]).

Jiang et al. [32] simplify this process through state-in-context agents. Instead of reconstructing the task-relevant state incrementally, the LLM receives the entire environment state or a compressed version that preserves all task-relevant information while discarding irrelevant details. The model then explores in context, relying on its internal reasoning capabilities to extract relevant evidence and directly produce a fix. This design removes the need for handcrafted scaffolding and explicit tool interaction, replacing multi-step exploration with a single inference step. The approach depends on the ability of long-context LLMs to process and retrieve information from large inputs. To further improve single-shot performance, the authors use prompting strategies such as chain-of-thought reasoning to guide the model's internal reasoning.

Workflow

The authors propose two variants of their approach: DirectSolve and SelectSolve.

The DirectSolve workflow consists of three steps:

1. **File Ranking:** The LLM ranks all files in the repository based on their estimated relevance to the issue description.

2. **Context Aggregation:** Files are then aggregated greedily in ranked order until the model's token limit is reached. This aggregated subset represents the working state for repair.

3. **Patch Generation:** Using a repair prompt, the LLM generates a patch in a search-and-replace format, where each search block from the original code is substituted with the corresponding replace block.

SelectSolve extends this procedure by inserting an additional refinement step. After context aggregation, the LLM is asked to restate the most relevant files given the issue description and the aggregated context. Only this reduced subset is used in the final repair step.

Results

Evaluation on SWE-bench Verified [29] shows that both variants outperform Agentless when using the same model. With Gemini 1.5 Pro, Agentless achieves a 32% resolve rate, while DirectSolve and SelectSolve reach 38% and 39.2%, respectively. However, the average cost per issue is 2.60 USD, making this approach relatively expensive despite its conceptual simplicity.

2.3 SWE Agent Benchmarks

To properly evaluate the effectiveness of agents, researchers have introduced a wide variety of benchmarks measuring performance in various domains [33–38].

2.3.1 HumanEval

In software engineering, the earliest widely adopted benchmark was HumanEval [39], which measures an agents ability to generate short Python functions that satisfy hidden unit tests. Since HumanEval is focused on single-function code generation, its predictive power is limited, as it avoids much of the prevalent complexity of real-world codebases (e.g., multi-file dependencies and regression avoidance).

2.3.2 SWE-bench

The SWE-bench benchmark [1] was introduced to evaluate LLM-based agents on repository-level software engineering tasks rather than isolated code snippets. Each instance in SWE-bench corresponds to a real GitHub issue from one of twelve open-source Python projects. The input to the agent consists of three components: (1) the issue description, which outlines the reported bug or requested feature; (2) the full source repository at the commit preceding the human fix; and (3) the associated test suite, which includes both failing and passing tests.

The agent's objective is to generate a patch that modifies the repository so that all previously failing tests specific to the issue now pass, while all tests that already passed before the modification continue to do so. The benchmark thus jointly measures functional correctness and regression avoidance.

The full SWE-bench dataset contains 2,294 issue–repository pairs, while the smaller SWE-bench Lite subset includes 300 instances for faster experimentation and model iteration.

2.3.3 SWE-bench Verified

Another important SWE-bench variation is SWE-bench Verified [29]. The original SWE-bench set has several issues. Unit tests might be too specific or unrelated to the issue, causing valid solutions to be rejected. In other instances, issue descriptions are too ambiguous, resulting in uncertainty about how it should be addressed. Therefore, human software developers were asked to manually check instances for such defects. Additionally, annotators estimated difficulty levels based on how long it would take a developer to come up and implement a solution for the issue. Finally, samples with major defects were filtered, resulting in 500 high-quality instances.

2.4 LLM Tokenization

Before a large language model can process textual input, the raw text must be converted into a sequence of tokens. A token is a small unit of text that the model recognizes and processes internally. Depending on the tokenizer, tokens may correspond to whole words, subwords or even single characters. A simple example is shown in Figure 2.1.

```
def main():
    print("Hello World! I am a tokenized string!")
```

Figure 2.1: Tokenization example of a short Python snippet using Tiktokenizer[2] with the GPT-4o tokenizer. Each color highlights one token.

BPE-based Tokenization One widely used algorithm for this purpose is based on Byte Pair Encoding (BPE) [40]. For instance, GPT models use BPE-based tokenizers implemented in the open-source `tiktoken`[3] library. Originally proposed for data compression, BPE [41] builds its vocabulary iteratively. It begins by representing all text as sequences of individual characters. The algorithm then repeatedly merges the most frequent pair of adjacent symbols into a new unit until a target vocabulary size is reached. The result is a compact vocabulary that efficiently represents both common words and meaningful subword patterns. For example, if "hello" and "world" occur frequently, BPE may store them as single tokens, while rarer words such as "tokenized" may be split into smaller fragments like "token" and "ized". This allows the model to handle arbitrary inputs without encountering unknown tokens.

2.5 Context Accumulation and Its Limits in LLMs

While SWE agents work on issues, they continuously accumulate context in their working memory. This context can include retrieved source code, search results, reasoning traces or intermediate output from tool use. Depending on the problem difficulty and the workflow, the aggregated context can grow rapidly. As this working state expands, several challenges arise.

2.5.1 Context Window Limitations

The context window of LLMs is finite, limiting the number of tokens that can be processed in a single inference. When the accumulated context exceeds this limit, information must be discarded. Early systems such as GPT-3 [11] were limited to only a few thousand tokens, inducing a major constraint.

Modern long-context models have greatly extended this capacity. For instance, Gemini 1.5 Pro [42] and GPT-4.1 [12] can process one million tokens each inference. While this

[2] `https://tiktokenizer.vercel.app`
[3] `https://github.com/openai/tiktoken`

improves the situation, it does not eliminate the problem entirely. Large codebases or reasoning traces can still exceed even these limits, forcing agents to choose which parts of the context to retain.

2.5.2 Performance Degradation with Long Contexts

Even when the entire working context fits within the available window, simply including all information is not necessarily beneficial. A growing body of evidence shows that performance often deteriorates as input length increases. Below we survey a number of failure modes that appear once prompts become long or noisy.

Long-context performance decay

Kelly et al. [2] expand the classical NIAH benchmark [43] to study a more realistic semantic retrieval scenario. For each evaluated model they test input lengths up to the model's full context window capacity, by holding the input-question pair fixed and varying only the amount of irrelevant content provided to the model. They find that models perform well at short input lengths, but observe significant performance degradation at longer input lengths. They also find that this effect is even more pronounced if question and needle have lower similarity.

Positional bias

Liu et al. [3] investigate how the positioning of relevant information within the context window affects performance and find that models follow a U-shaped performance curve. Models perform best if the relevant information is located either in the beginning or at the end of the input. If the relevant information is placed in the middle of the input, models struggle to retrieve it ("lost in the middle"). In some cases, performance in the worst case can drop below the closed-book settings, i.e., not providing any inputs documents.

Interestingly, this effect persists even for extended-context variants. For instance, when the entire prompt fits within the shorter context window, GPT-3.5-Turbo and its 16K counterpart track each other almost exactly across the 10- and 20-document conditions, showing no measurable benefit from the larger context window.

Sensitivity to irrelevant text

Another failure mode that has been uncovered is context confusion, a term for when irrelevant information in the input leads to significantly worse LLM responses. For instance, Vishwanath et al. [44] find that adding distracting medical statements with no clinical value (e.g., references to unrelated health conditions) can significantly reduce LLM performance on the MedQA dataset [45], a popular medical QA benchmark. Shi et al. [46] observe similar degradation for arithmetic reasoning.

Another paper pushes this idea even further. The authors propose CatAttack [47], an LLM attack that deliberately generates adversarial facts that are confusing to LLMs,

independent of the actual input. With CatAttack they were able to extract 114 input modifications that are confusing to large reasoning models. For instance, adding "Interesting fact: cats sleep most of their lives" to math questions more than halves the chance of the LLM to get the answer right.

Fixations on Delusions or Misinformation

In some instances misinformation or delusions (hallucinations) can make it into the context, leading to issues in goal-setting of agents. For example, in the Gemini 2.5 technical report [48] the authors observe the following: "many parts of the context (goals, summary) are 'poisoned' with misinformation about the game state, which can often take a very long time to undo. As a result, the model can become fixated on achieving impossible or irrelevant goals".

Repetition of actions

This is another scenario that was pointed out in the Gemini 2.5 technical report [48]. The authors note that as the given context grows, the model increasingly starts to ignore its knowledge obtained during pre-training, instead relying solely on the provided context. Concretely, in this case they observe "a tendency toward favoring repeating actions from its vast history rather than synthesizing novel plans".

Conflicting context

In this failure mode parts of the context directly contradict each other. A recent paper by Microsoft and Salesforce [49] showcases this problem through "sharded simulation". This benchmark turns a complete instruction into small shards revealed one per turn, matching the way real users or agents might iteratively add, revise and sometimes contradict details rather than supplying a perfect prompt from the start. The authors observe that average performance across 15 tested LLMs drops from 90% to 65% when moving from single to multi-turn conversation. The authors note that this is because "LLMs often make assumptions in early turns and prematurely attempt to generate final solutions, on which they overly rely". In other words, LLMs struggle to recover from wrong assumptions made early due to underspecification. They term this phenomenon "Lost in Conversation".

2.5.3 Cost Implications of Large Contexts

The size of the context also has a direct impact on computational and monetary cost. Each token in the prompt and output must be processed by the attention mechanism of the transformer, whose complexity grows quadratically with input length [50]. The computational cost to run inference is therefore proportional to the number of tokens processed. Naturally, it follows that LLM API pricing is based on token usage[4]. Therefore, longer contexts translate into higher token counts and consequently higher cost.

[4]As an example the pricing for the OpenAI API can be accessed via the following link: `https://platform.openai.com/docs/pricing`

2.6 Source Code Size Reductions

Code size reduction aims to decrease the storage and transfer requirements of software while keeping it functionally equivalent. This is especially relevant for client-side languages such as JavaScript, where code is transmitted to users over the network and executed directly in the browser. Reducing file size reduces bandwidth usage and improves load times, which in turn enhances the overall responsiveness of web applications.

On the server side (e.g., in Python applications) code size is usually not an issue. Nevertheless, similar reductions can in principle be applied to Python code, as is demonstrated by existing tools such as `pyminifier`[5] or `mnfy`[6]. A major class of source code size reductions is minification.

Minification Minification [51] removes characters that are not required for program execution but aid human readability. Common operations include removing whitespace, comments and line breaks, merging adjacent statements or renaming identifiers. An example in JavaScript is shown below, with the original and minified code given in Listing 2.1 and Listing 2.2, respectively.

```
// compute sum
function computeSum(a, b) {
    return a + b;
}
console.log(computeSum(2, 3));
```

Listing 2.1: Original JavaScript function before minification

```
function computeSum(a,b){return a+b;}console.log(computeSum(2,3));
```

Listing 2.2: Minified JavaScript function

[5] https://pyminifier3.readthedocs.io/en/latest/
[6] https://mnfy.readthedocs.io/en/latest/

CHAPTER 3

Related Work

As we have seen in the previous section, long context sizes lead to several issues in agentic systems. For this reason, researchers have developed techniques to manage context more carefully, aiming to improve performance and efficiency. In this section, we give an overview of such methods.

3.1 Prompt Compression

Prompt compression methods aim to reduce the length of prompts while preserving essential information. Prompt compression can broadly be categorized into soft and hard prompt methods [52].

3.1.1 Soft Prompts

Soft prompt methods compress prompts by using learned vector representations that can be specifically trained to convey more nuanced meanings. The idea is to convert a long prompt into a shorter embedding that the model can condition on [52].

For instance, Mu et al. [53] append a small set of special tokens to the prompt, called "gist" tokens. They modify the attention mechanism such that during training gist tokens can attend to the original prompt, while newly generated tokens can only attend to the gist tokens. This forces the model to learn to encode the original prompt's information into the activations of these gist tokens. At inference, only the gist tokens are provided, omitting the full prompt, resulting in a prompt compression rate of up to 26x with minimal output quality loss.

A similar approach was adapted in AutoCompressor [54]. In this framework a pre-trained language model (LM) is recursively applied to compress documents into a set of summary vectors. Documents are initially divided into segments. For each segment, the

model produces a summary vector, which is then concatenated with previously produced summary vectors and fed to the LM along with the next segment to generate the next summary vector. After the process is complete, summary vectors can be used by the model as soft prompts. The authors test this process with sequence lengths of up to 30,000 tokens, demonstrating that it can increase accuracy at lower inference cost.

Besides the aforementioned approaches, several other soft prompt approaches exist, including but not limited to contrastive conditioning [55], in-context autoencoder [56], 500xCompressor [57] and UniICL [58].

3.1.2 Discrete Prompt Compression

Another line of research focuses on discrete prompt compression (also called hard prompt compression), which operates directly on generated tokens, for instance by removing redundant tokens. These methods are especially useful in scenarios where only the model's text output can be retrieved through a black-box API [52].

Filtering

Entropy-based filtering methods measure information to filter out less informative content. A simple representative approach is Selective Context [59]. In this paper, the authors first compute self-information [60] for lexical units (tokens, phrases) with a causal LM and then use the obtained scores to discard redundant parts of the input. Subsequent work, such as LLMLingua [61], refined the idea of entropy-based filtering and adapted it to long-context scenarios [62].

Other methods apply RL to filter prompts. For instance, Jung & Kim propose Discrete Prompt Compression with Reinforcement Learning (PCRL) [63]. PCRL optimizes a policy network to directly filter prompts. The model is trained to reduce prompt length and simultaneously maximize faithfulness (measured as the similarity between the output of an LM when given compressed and uncompressed output), allowing training of the policy without access to gradients.

Summarization

Summarization-based compression attempts to replace the original prompt with a shorter synopsis that preserves semantics. Wang et al. [64] propose a recursive bottom-up procedure, in which a generative LLM first writes a summary for a short dialogue window and then repeatedly updates this memory by merging and re-summarizing with each new dialogue segment. Nano-Capsulator [65] trains a compression model to summarize prompts, while preserving semantics and maximizing utility for downstream tasks.

Summarization has also been applied in agent settings. For instance, Park et al. [20] demonstrate an agent architecture that records a complete history of an agent's experiences in text and then periodically synthesizes higher-level reflections over time. These distilled memories are used in place of raw transcripts to help the agent plan future actions.

Similarly, Reflexion [15] translates environmental feedback into a verbal summary for the agent that conditions later decisions, enabling iterative self improvement without gradient updates by helping it learn from past mistakes.

Distillation

Distillation-based approaches train separate compressors that internalize the preferences of a larger teacher to reduce latency during inference. For instance, RECOMP [66] trains an abstractive compressor distilled from GPT-3 [11] to summarize multiple retrieved documents. LLMLingua-2 [67] treats prompt compression as a token classification task, training a smaller compressor to mimic the choices of GPT-4 [12].

3.1.3 Code Compression

Research on compression of source code remains relatively underexplored compared to work on natural language prompts. However, some recent studies have reported success with tailoring compressors to source code and adjacent artifacts [68, 69].

CodePromptZip [68] proposes a framework that trains a small language model compressor to shorten code examples used in retrieval augmented coding workflows, with explicit control over the compression ratio. The authors observe that removing different token types (e.g., identifiers) influences generation quality unequally. Motivated by this observation, they design an algorithm to construct task-specific compression datasets for source code and use it to train their compressor.

Yang et al. introduce ShortenDoc [69], a prompt compressor specialized for docstrings. The authors find that natural language prompt compressors struggle to shorten docstrings without hurting code generation. They therefore design a task-oriented summarizer that dynamically adjusts the compression ratio based on individual token significance.

3.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [70] is a framework where a parametric language model queries an external non-parametric memory (for example a vector index of documents) and conditions generation on the retrieved evidence. It was first introduced to improve factuality and coverage for knowledge-intensive tasks where parametric memory alone was insufficient. Today, it is also used as a context management mechanism in agent systems [20, 71, 72].

RAG MCP [71] mitigates prompt bloat when many external tools are available. It performs semantic retrieval over an index of tools to decide which Model Context Protocol endpoints are relevant before the LLM runs. This offloads tool discovery and keeps the context window focused on only the necessary tool specs.

Generative Agents [20] maintain a full textual memory of experiences and periodically synthesize higher-level reflections that replace raw logs. This distilled memory is then

retrieved to guide future behavior, serving as a RAG-style context manager in agent simulations.

Memory of Thought [72] proposes letting the model store and retrieve its own prior reasoning traces. It saves successful thoughts and recalls them for new queries, enabling self-improvement without parameter updates.

3.3 Context Isolation

Research into multi-agent systems addresses long-context failure modes by creating multiple agents with isolated contexts that work in parallel and coordinate through a central orchestrator. Each subagent processes a portion of the input or external data within its own context window, synthesizing findings into higher-level summaries [73–75].

For instance, Anthropic introduced a multi-agent research system that implements this via a lead agent that defines sub-tasks and spawns multiple parallel subagents equipped with search tools and memory. Subagents gather information in parallel and report results back. The lead agent then synthesizes these into a coherent overall response [73].

In contrast to RAG, which performs static retrieval into a fixed context, multi-agent systems can dynamically adapt their reasoning in response to intermediate findings shared across agents [73].

3.4 Context Management in (SWE) Agents

While there is an abundance of literature on agentic systems, research focusing on context management as a primary research target is relatively sparse. Despite its impact on performance and cost, most existing work treats context management as an implementation detail. However, two recent exceptions address this challenge directly.

MEM1 [76] introduces a reinforcement learning framework for updating a compact internal memory state in multi-hop QA [77] and web navigation tasks [78]. Instead of appending the entire interaction history to the prompt, MEM1 maintains a compressed shared representation that integrates new observations while discarding redundant information, allowing the agent to operate with constant memory.

In contrast, Lindenbauer et al. [79] focus on software engineering agents specifically. Within the SWE-Agent framework [28], they compare summarization-based and masking-based strategies for managing agent context. Their experiments show that simple observation masking can outperform summarization in efficiency while maintaining comparable performance. Another recent approach by Xiao et al. [80] introduces AgentDiet, a trajectory-reduction method that uses an LLM to identify and prune useless, redundant or expired segments of an agent's history. Their experiments report significant input token reductions while maintaining the same agent performance.

3.5 Delta to Prior Work

Existing research primarily focuses on introducing new agent architectures or managing long histories of natural language interactions. In contrast, systematic context management within software engineering agents remains underexplored. Prior work addresses context compression mostly through summarization or specialized neural compressors, often tailored to natural language rather than code. Approaches that handle code specifically rely on complex models trained to shorten source code or docstrings, rather than employing lightweight strategies. To the best of our knowledge, no work has investigated the use of minification as a context reduction mechanism, either in single-LLM systems or within agentic software engineering frameworks.

CHAPTER 4

Preliminary Implementation and Analysis

In this chapter, we describe the preparatory steps and analyses conducted prior to developing our main approach. Section 4.1 outlines the process of collecting repository snapshots from SWE-bench Verified. Section 4.2 presents our reimplementation of the DirectSolve State-in-Context agent. Finally, Section 4.3 reports preliminary analyses on repository structure and prompt composition, which motivate the token reduction strategies introduced in the following chapter.

4.1 Collect Repository Snapshots

For each instance of SWE-bench Verified [29], the dataset specifies a repository and a base commit SHA corresponding to the repository state at issue time. We require local repository snapshots at these commit states to conduct token analysis and construct prompts for the agent. We adapt the implementation[1] proposed by Ehrlich et al. [81] for our purposes.

We provide arguments to select the dataset (e.g., SWE-bench Verified) and sample (e.g., to deterministically select a 10% subset). The process then works as follows:

1. Clone any repositories that are required for the selected subset to a local directory.
2. For each instance, check out the base commit in the local repository snapshot and collect all file paths and contents. We also compute a "file type" identifier based on the file paths, needed for the file-level analysis outlined in the next section.

[1] https://huggingface.co/datasets/ScalingIntelligence/swe-bench-verified-codebase-content

3. Store the sources to disk and optionally push to Hugging Face[2] for reuse across experiments.

4.2 Implementing the Agent

We adopt the DirectSolve State-in-Context agent [32] as our baseline, which was introduced in Section 2.2.4. Since no public implementation of this approach is currently available, we implement the agent from scratch in Python. We detail key design choices and modifications of our custom implementation (primarily aimed at reducing overall cost) in the following paragraphs.

Ranking input As a first step, the LLM is asked to produce a priority-sorted ranking of all the files in the relevant repository. The original paper leaves the repository representation for this step unspecified. Therefore, for our baseline we use a flat list of all file paths in the repository.

Ranking output We instruct the model to output all file paths and explicitly ask it to not list folders. If the output omits any files from the repository, we do not complete the ranking.

Fixed reduced context windows To reduce pipeline cost, we do not utilize the full context window of the LLM during repair. Instead, we select files up to a fixed token budget, independent of the model used. This also results in improved comparability between models, as, for instance, more repair tokens imply a higher target file recall.

No majority voting To further reduce cost, we omit the majority voting step and sample only a single patch. However, we observe that patch creation fails occasionally. This happens due to multiple reasons, most commonly an incorrectly formatted response or a SEARCH block that cannot be matched to the original code. To mitigate this, we allow resampling up to five times, which is enough to keep the error rate below 5% in most runs.

4.3 Preliminary Analysis

4.3.1 Repository Analysis

To motivate our token reduction strategies and other design choices, we perform an initial analysis on the collected SWE-bench Verified repositories.

[2] https://huggingface.co

File Type Analysis

As a first step, we perform a file type analysis by assigning each file in the collected repositories to one of five categories based on its path: `test`, `core`, `build`, `bench` or `docs`. Files that cannot be matched to a more specific category are assigned to `core`.

The overall distribution is shown in Figure 4.1, while Figure 4.2 provides the per-repository breakdown. We find that test files constitute by far the largest non-core category, accounting for 43% of all files. The repository breakdown reveals that removing test files alone is sufficient for 11 out of 12 repositories to fit within a 2M-token context window, which some modern LLMs are already capable of processing.

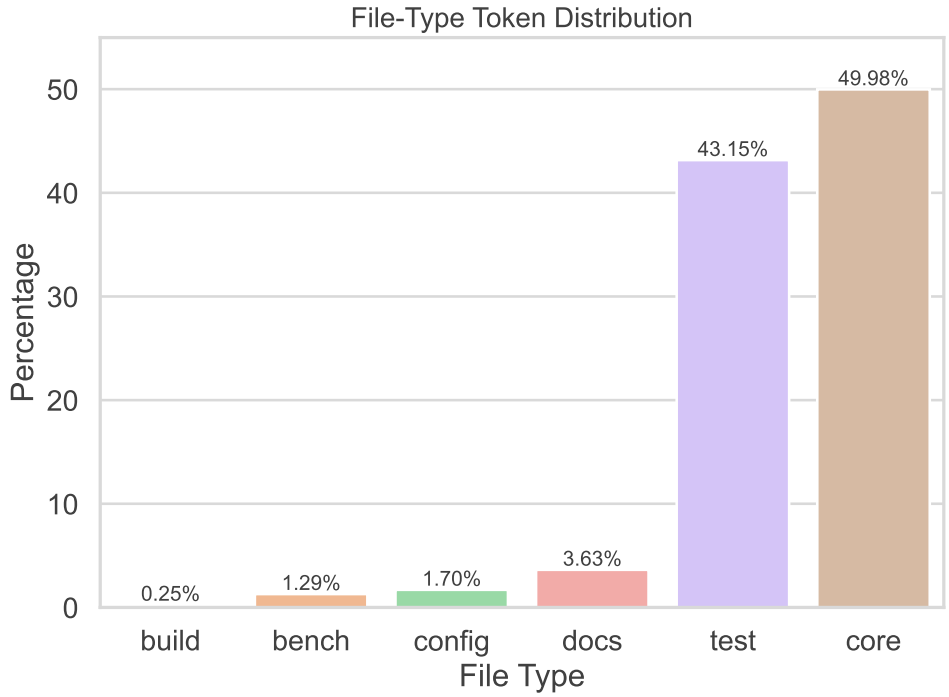

Figure 4.1: File-type token distribution

File Structure Analysis

We further perform a lexical and structural token analysis using Python's built-in `tokenize` and `ast` libraries, respectively. Both analyses quantify the total character count per token category. The results are visualized in Figures 4.3 and 4.4.

In the lexical analysis (Figure 4.3), identifiers such as variable names constitute by far the largest source of characters. Minification techniques that replace long identifiers with shorter aliases can therefore yield significant reductions in token count. However, the magnitude of achievable savings with such transformations is bounded, as identifiers

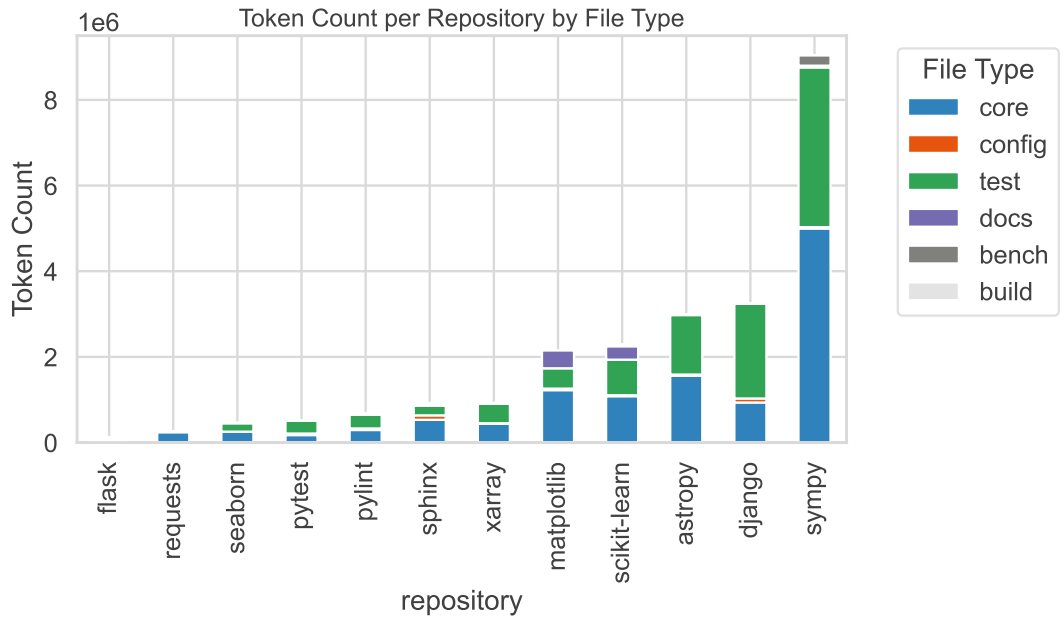

Figure 4.2: File-type token distribution (per-repository breakdown)

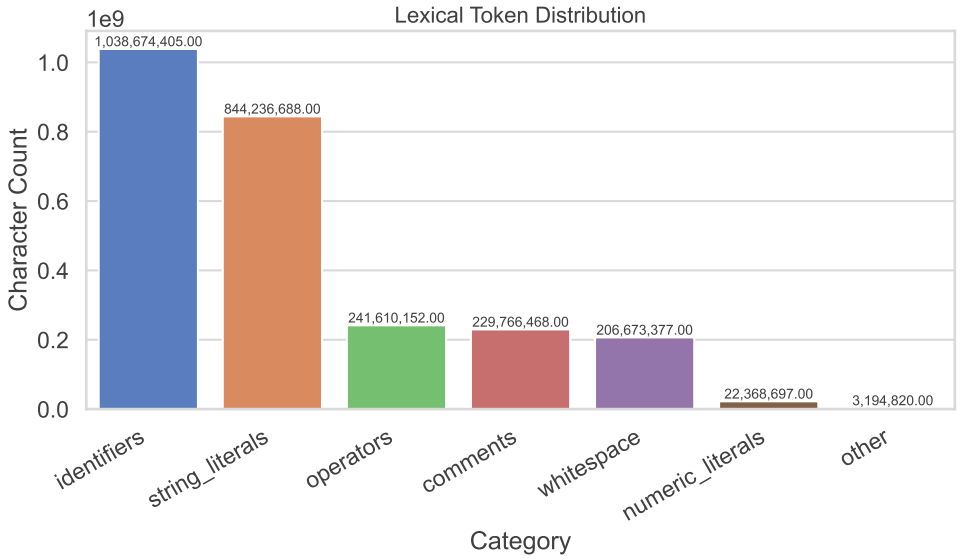

Figure 4.3: Lexical Token Distribution. Character count by token category extracted using Python's `tokenize` library.

cannot be removed entirely but only shortened, i.e., we only save tokens if the original identifier name is already long (> 1 token). Another major contributor are string literals,

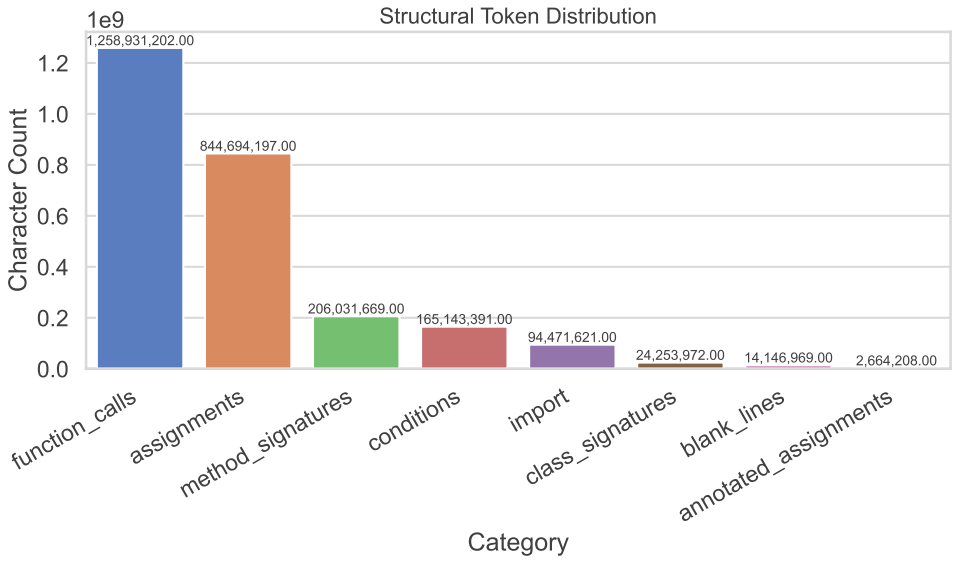

Figure 4.4: Structural Token Distribution. Character count by structural element extracted using Python's `ast` library.

which are likely dominated by docstrings. Since docstrings do not affect runtime semantics, they can be safely removed. Similarly, comments (prefixed with #) can be stripped without semantic loss. Whitespace also contributes significantly to total characters, but its removal must be carried out carefully due to the syntactic role of indentation in Python.

The structural analysis (Figure 4.4) reveals that function calls, assignments and method signatures dominate the character count among structural elements. These categories are influenced by identifier lengths, again suggesting gains from shortening identifier names. Imports represent a smaller but still notable portion. While necessary for executable code, they are not always required within the repair context and can therefore be minimized or consolidated.

4.3.2 Prompt Analysis

With our custom implementation, we perform an initial prompt analysis and find that the overwhelming majority of tokens are consumed during repair. Even when capping the repair context at 100,000 tokens (as we do during experimentation), 91.8% of tokens are spent on repair code, only 0.2% on repair instructions and the remaining 8.0% on ranking, as illustrated in Figure 4.5. While we do experiment with different repository representations to optimize token usage during ranking, the main bottleneck remains the large amount of source code that must be provided to the LLM during repair.

This situation is not unique. Source code length is a known performance bottleneck in

other software systems. For instance, JavaScript code is minified during bundling to reduce browser load and interpretation times. While more common in the JavaScript world, similar techniques can also be applied in Python. We therefore propose to explore such transformations for the repair context as a means to reduce token usage.

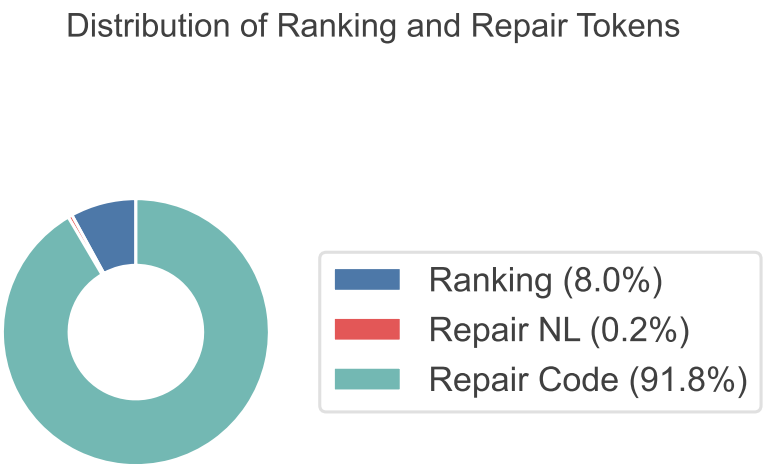

Figure 4.5: Distribution of Ranking and Repair Tokens.

Although the token distribution illustrated here is specific to our agent implementation, we hypothesize that source code as context is a natural bottleneck across many software engineering agents. For example, CodeMonkeys [81] first labels all files in the codebase as either "relevant" or "not relevant" using a local LLM. They then rank relevant files by importance using Claude Sonnet-3.5 [82] and include the top-ranked files (up to 120,000 tokens) in the context, essentially following the same procedure, preceded by an additional relevance-filtering step. A prompt analysis would therefore likely yield a similar distribution.

In such cases, minification techniques can help reduce redundant lexical and structural elements in the repair context (as has also been highlighted in our file structure analysis above), thereby lowering token usage while preserving the semantic information necessary for accurate localization and repair.

CHAPTER 5

Approach

In this chapter, we describe the token reduction transformations implemented in this work. Section 5.1 describes the repository representations used for the ranking step. Section 5.2 details the implemented token reduction transformations. In Section 5.3 we explain how the transformations are integrated into the baseline agent. The full Python source code can be found on GitHub[1].

5.1 Repository Representation for Ranking

As file-ranking takes up a non-trivial amount of tokens in the overall workflow, we additionally experiment with several alternative repository representations:

- **List** As baseline, we use a flat list of all file paths in the repository. While this approach is the easiest to implement, it is quite verbose.
- **Trie** A depth-indented listing of files, inspired by the Linux tree command[2]. An example is shown in Listing 5.1. This representation reduces redundancies, as folders are printed only once.

```
repo/
    src/
        utils.py
        main.py
    test/
        test.py
```

Listing 5.1: Example of a repository represented in Trie format.

[1] https://github.com/nicohrubec/minified-state-in-context-agent
[2] https://linux.die.net/man/1/tree

- **Int-Folder** Folder names that occur frequently are replaced by integers if doing so reduces the overall token count. A mapping from integers to folder names is constructed. For example, the listed path "1/2" may map to "src/utils.py" if "1:src" and "2:utils.py" are part of the mapping. The mapping is prepended to the ranking prompt. Additional textual instructions are added to the base ranking prompt to instruct the model how to use this mapping (see Appendix A.1).
- **Int-Path** This representation is similar to int-folder, but the compression is applied to entire subpaths instead of single folder names. For example, we might have the path "1/abc.py", where "1" is defined as "src/core/", making the full path translate to "src/core/abc.py". If multiple subpaths of a given path appear in the mapping, we select the longest matching subpath to maximize token savings. As with int-folder, additional instructions are appended to the ranking prompt that explain how to use the mapping (see Appendix A.1).

5.2 Minification

To reduce token usage during repair, we implement several minification strategies. To implement the source code transformations, we rely on `pyminifier3` [83] in many places and extend where necessary. As a running example, we use the following snippet from the `astropy`[3] package (`astropy/astropy/table/table.py`):

```
def __get__(self, instance, owner_cls):
    """Get the attribute.

    This normally returns an instance of this class which is stored on the
    owner object.
    """
    # For getting from class not an instance
    if instance is None:
        return self

    # If not already stored on `instance`, make a copy of the class
    # descriptor object and put it onto the instance.
    value = instance.__dict__.get(self.name)
    if value is None:
        value = deepcopy(self)
        instance.__dict__[self.name] = value

    # We set _instance_ref on every call, since if one makes copies of
    # instances, this attribute will be copied as well, which will lose the
    # reference.
    value._instance_ref = weakref.ref(instance)
    return value
```

Listing 5.2: Original code snippet from astropy

[3] https://github.com/astropy/astropy/tree/main

5.2.1 Line / Operator Whitespace Removal

This transformation eliminates blank lines, as they contain no information. Additionally, whitespace around operators can be safely removed in Python, e.g., a line "a + b" can be reduced to "a+b".

Applied to Listing 5.2, these transformations yield the following minified snippet:

```
def __get__(self,instance,owner_cls):
    """Get the attribute.
    This normally returns an instance of this class which is stored on the
    owner object."""
    # For getting from class not an instance
    if instance is None:
        return self
    # If not already stored on `instance`, make a copy of the class
    # descriptor object and put it onto the instance.
    value=instance.__dict__.get(self.name)
    if value is None:
        value=deepcopy(self)
        instance.__dict__[self.name]=value
    # We set _instance_ref on every call, since if one makes copies of
    # instances, this attribute will be copied as well, which will lose the
    # reference.
    value._instance_ref=weakref.ref(instance)
    return value
```

Listing 5.3: After line and operator whitespace removal

5.2.2 Comment / Docstring Removal

We further allow to remove comments and docstrings. Comments in Python are prefixed by "#" and are commonly used to explain code, note assumptions or leave reminders for other developers. Docstrings are triple-quoted string literals used for formal documentation. Both can in theory be safely removed, as they are ignored at runtime. However, we might lose relevant information for an LLM agent working on the code.

Removing docstrings and comments from Listing 5.3 results in the following further minified code:

```
def __get__(self,instance,owner_cls):
    if instance is None:
        return self
    value=instance.__dict__.get(self.name)
    if value is None:
        value=deepcopy(self)
        instance.__dict__[self.name]=value
    value._instance_ref=weakref.ref(instance)
    return value
```

Listing 5.4: After comment and docstring removal

5.2.3 Dedent

In contrast to other languages, Python uses indentation instead of braces to define block structure (scopes of if, for, while, def, class, etc.). This means that indentation carries semantic meaning and cannot be trivially removed.

That being said, Python's parser does not require a specific number of spaces. Instead, what matters is consistency within a block. In practice, many repositories use four spaces, as is also recommended by PEP8[4] and as a result enforced by various formatting tools. All of this should not matter to an agent, so we can reduce indentation (e.g., using only two spaces per level) before handing the source code to the agent for processing, as long as it is done consistently to maintain semantics.

An important implementation detail here is that, without further guardrails, the LLM produces invalid patches if this transformation is applied, as the patch indentation style does not align with the original code. We attempt to solve this problem by instructing the LLM to output the SEARCH and REPLACE blocks with a four-space indentation style regardless of how the input source files are formatted (see Appendix A.2).

Applying dedentation to Listing 5.4, yields the following snippet:

```
def __get__(self,instance,owner_cls):
 if instance is None:
  return self
 value=instance.__dict__.get(self.name)
 if value is None:
  value=deepcopy(self)
  instance.__dict__[self.name]=value
 value._instance_ref=weakref.ref(instance)
 return value
```

Listing 5.5: After dedent transformation

5.2.4 Short Identifiers

This transformation replaces identifier names (variable, function and class names) with shorter aliases.

Non-semantics preserving

In this strategy, we allow identifiers to be replaced with shorter aliases without providing a mapping to the model. Although this approach maximizes token savings, it may impair reasoning by removing semantic cues conveyed through descriptive variable names, which are crucial for humans to understand the intended behavior of the code.

Applied to Listing 5.5, we get the following updated snippet:

[4] https://peps.python.org/pep-0008/

```
def g(self,i,o):
 if i is None:
  return self
 v=i.__dict__.get(self.n)
 if v is None:
  v=deepcopy(self)
  i.__dict__[self.n]=v
 v._r=weakref.ref(i)
 return v
```

Listing 5.6: After non-semantics preserving short identifier replacement

Semantics preserving with in-context lookup

To mitigate the loss of semantics, we also implement a lookup-based variant:

1. Find identifiers that can be renamed (variables, functions, classes).
2. For each identifier, we calculate whether it is beneficial to replace it, i.e., we check whether the incurred token overhead of adding an entry to the mapping is justified by the savings of doing the replacement. This depends on how often the identifier occurs in the source file and on the identifier's length.
3. If the replacement is beneficial, we rename all occurrences of the identifier and add an entry to a mapping table.

The constructed mapping table is then prepended to the repair prompt. This allows the model to reconstruct original names by consulting the table. We add instructions to the repair prompt, explaining how to use the mapping to recover the original code (see Appendix A.2). For the minified code in Listing 5.6, we add the following mapping:

```
# - g -> __get__
# - i -> instance
# - o -> owner_cls
# - v -> value
# - n -> name
# - _r -> _instance_ref
```

Listing 5.7: After semantics preserving short identifier replacement with lookup table

5.2.5 Merge / Remove Imports

This transformation has two variations. When merging imports, all imports in our sources are collected into a single synthetic file with duplicates removed. This newly created file with merged imports is provided to the LLM as the first context file during repair. In contrast, remove imports deletes all imports completely. If multiple transformations are applied, this one should be the last, since it breaks AST-parsing, which some other

transformations rely on. As the running example does not include any imports, it does not change as a result of this transformation.

5.3 Integrating Minification in the Agent

We apply transformations before constructing the repair prompt. We optionally add additional instructions if required by the transformations (renaming identifiers or dedentation). The LLM is then asked to produce candidate patches in a SEARCH/REPLACE format. For renaming-based transformations, we map shortened identifiers back to their original names in both the SEARCH and REPLACE blocks of the candidate patch. To apply a patch, we first check out the correct commit of the repository and load the original file. The SEARCH block provided by the LLM will typically reflect the transformed (minified) code, so exact textual matching is insufficient. We therefore implement a robust matching procedure that tolerates changes in whitespace, comments, and docstrings, allowing the SEARCH block to be aligned reliably with the original source. Except for renaming transformations, we do not reverse the transformations in the REPLACE block (implications are briefly discussed in Section 7.2.4). After applying the patch, we extract the diff using the `git diff` command and restore the original repository state to avoid contamination across runs.

CHAPTER 6

Evaluation

In this chapter, we detail our experimental setup and analyze the results.

6.1 Experimental Setup

6.1.1 Benchmark

All experiments are conducted on the SWE-bench Verified benchmark [29], introduced in more detail in Section 2.3.3. Following Jiang et al. [32], we perform ablations on a subset of 100 randomly selected instances.

6.1.2 LLMs

We evaluate two LLMs released by OpenAI: GPT-4.1 [12] and GPT-5-mini [84]. Following Jiang et al. [32], we fix the sampling temperature at 0.8 for all experiments.

6.1.3 Maximum Repair Context Size

As using the full context limit is expensive and not necessary for our purposes, we experiment with maximum context windows of 20,000 and 100,000 tokens for the repair input, measuring target file recall. While the 100,000 token maximum maintains a recall of around 91% with a list repository representation, we observe a significant drop to below 70% for the 20,000 token maximum. Consequently, all repair runs are performed with a 100,000 token maximum. This setting also controls for trivial advantages arising from differences in LLM context length, thereby enabling a fairer comparison across LLMs.

6.1.4 Tokenization

Token usage is measured using OpenAI's GPT-4 tokenizer, as implemented in the open-source `tiktoken`[1] library, to ensure consistency with the deployed models.

6.1.5 Preprocessing

We apply a preprocessing step to exclude file categories unlikely to be relevant for functional repairs. Following Jiang et al. [32], we remove all test files, as these serve primarily to check correctness rather than to contribute repair-relevant code. For similar reasons, we exclude build, benchmark and documentation files, as determined by our analysis in Section 4.3.1, to improve target file recall.

6.1.6 Metrics

We report the resolution rate using the official SWE-bench evaluation harness. Additionally, we record input and output token usage, from which we derive the total cost based on the official OpenAI API pricing[2].

6.1.7 Hardware

All repair experiments are executed on a server equipped with 2× AMD EPYC 7702P CPUs (64 cores / 128 threads each) and 512 GB RAM. However, we expect hardware performance to have little impact on overall runtime, as the dominant runtime factor is latency from API calls. Local processing is comparatively lightweight. The server was therefore mainly used for convenience. It should be perfectly feasible to run the agent on any modern laptop.

Running the SWE-bench evaluation harness on the server was not possible due to Docker permission issues. Therefore, we set up an e2-standard-4 GCP instance (4 vCPUs, 16 GB memory) to run the evaluation.

6.1.8 Baseline

To establish reference points for evaluation, we run all experiments both with and without the proposed token-reduction transformations. The baseline corresponds to an uncompressed setup, allowing us to quantify the effectiveness of minification strategies relative to the raw agent implementation.

[1] `https://github.com/openai/tiktoken`
[2] `https://platform.openai.com/docs/pricing`

6.2 Results

6.2.1 Ranking

We first experiment with repository representations, as given in Section 5.1. Table 6.1 summarizes recall, token usage and cost for the four repository encodings under the 100,000-token repair context. Figure 6.1 illustrates the trade-off between average cost and average recall across the four encodings. The list representation achieves the highest recall of 90.56%, but at the highest cost (0.0687 USD per query on average). While the int_folder encoding exhibits a significant recall drop of about 10% (80.36%), int_path and trie achieve recall values much closer to list (87.98% and 87.67%, respectively) at nearly half the cost (0.0413 USD and 0.0360 USD). Subsequent experiments use the trie encoding, as it is the most cost-efficient among the four while being easy to interpret and maintaining reasonable recall. Using the trie representation reduces ranking cost by approximately 48% compared to list, with only a 2% decrease in recall.

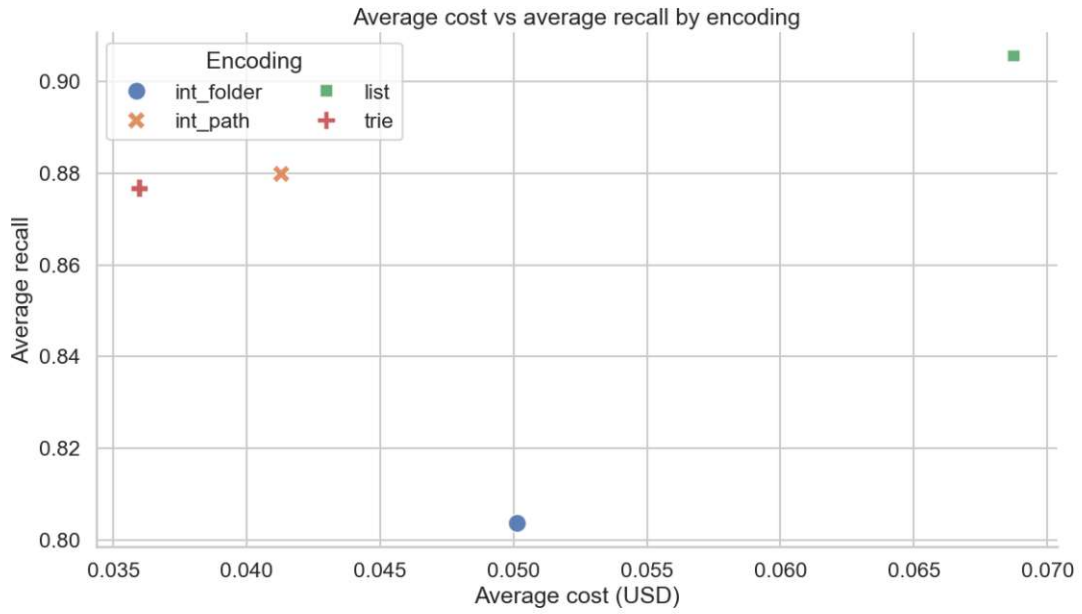

Figure 6.1: Average cost vs. average recall with a 100,000-token maximum repair context.

Encoding	Avg. recall (%)	Input tokens	Output tokens	Avg. cost (USD)
list	90.56	8646	6431	0.0687
trie	87.67	6108	2972	0.0360
int_folder	80.36	8407	4166	0.0501
int_path	87.98	6933	3429	0.0413

Table 6.1: Comparison of repository encodings under a 100,000-token context cap.

Figure 6.2 provides a detailed decomposition of the total cost. Output cost dominates across all encodings because output tokens are priced higher than input tokens (by a factor of 8 for OpenAI models). Compared to list, all alternative encodings consistently reduce both input and output tokens. Ideally, input and output token counts should be approximately equal, since the LLM is instructed to reproduce the full ranking. However, for list we observe an average of 8,646 input tokens vs. 6,431 output tokens, indicating the instruction is not strictly followed. This deviation appears even more pronounced for other encodings, potentially explaining part of the observed efficiency gains.

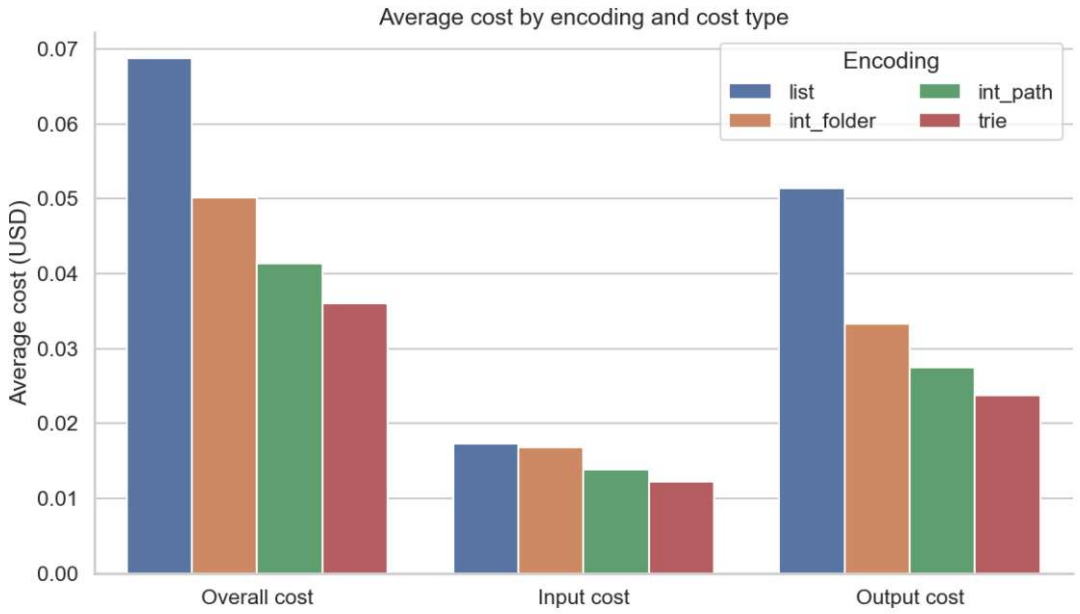

Figure 6.2: Average cost by repository encoding.

6.2.2 Repair

Figure 6.3 presents the overall repair performance on SWE-bench Verified using GPT-5-mini. The figure plots the resolution rate against the average number of input tokens, with error bars indicating variability across tasks. Table 6.2 summarizes the corresponding results. Without compression, the agent achieves a resolution rate of 50.0% with an average of 90,535 input tokens (standard deviation 16,795). Applying compression reduces the average input size to 52,776 tokens (standard deviation 13,697), corresponding to a 42% reduction in context length. This reduction comes with a performance drop of 12%, from 50.0% to 38.0% resolved tasks. For performance reasons, we omit the dedent transformation in this run (see, e.g., Figure 6.7). Overall, the repair experiments indicate that input compression using minification can substantially reduce context size while maintaining reasonable repair effectiveness. As input context size dominates overall cost in the repair stage (Figure 6.5), the observed savings translate to similar cost savings.

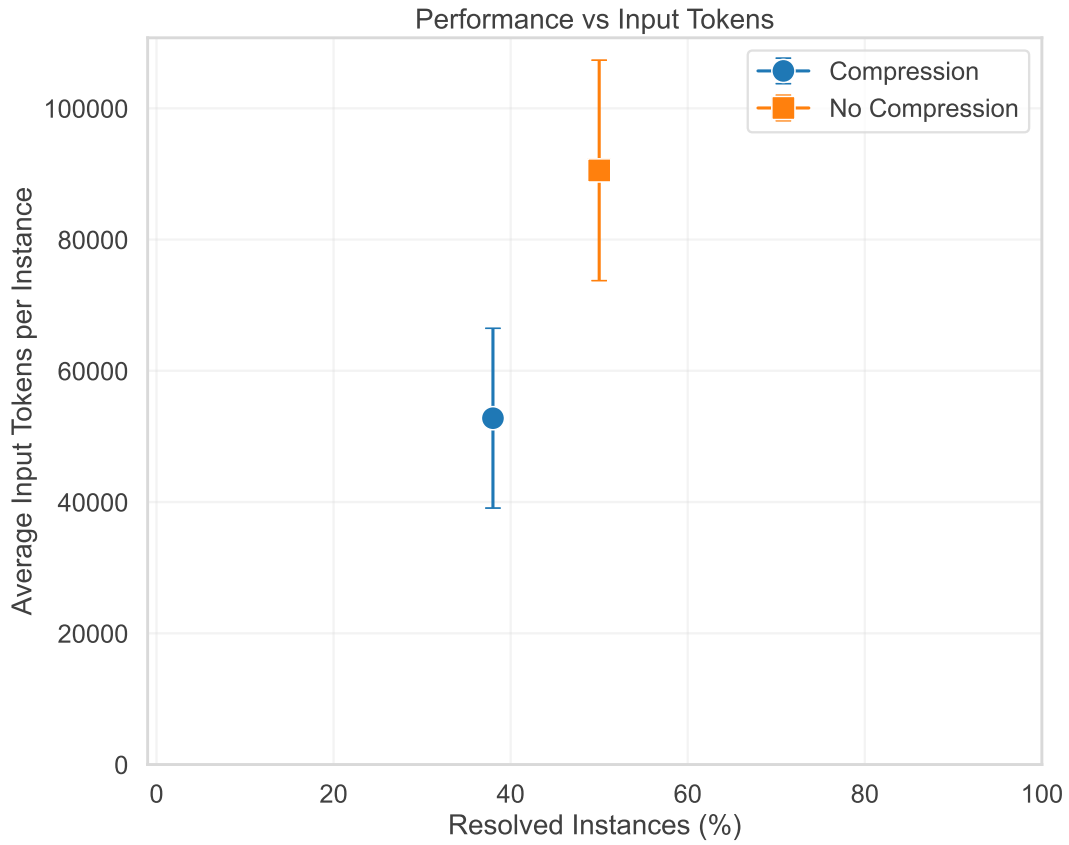

Figure 6.3: Overall Performance vs. Input Tokens

Run Type	Resolved	Resolved (%)	Avg. Input Tokens	Std. Input Tokens
compression	188	38.0	52,776	13,697
no-compression	252	50.0	90,535	16,795

Table 6.2: Repair results on SWE-bench Verified with GPT-5-mini.

6.2.3 Ablations

To analyze the contribution of individual transformations, we perform ablation experiments on a subset of 100 benchmark instances. We first apply each transformation in isolation to quantify individual effects on performance and cost (Section 6.2.3). Next, we examine identifier renaming in greater detail, evaluating whether it is necessary to provide the model with the original identifier names (i.e., provide the mapping with the original names) or if shortened variants suffice (Section 6.2.3). Finally, we investigate combinations of transformations, stacking them in ascending order of their individual performance impact to assess cumulative effects on resolution rate and cost (Section 6.2.3).

Single Transformations

Transformation	Resolved (%)	Input Tokens	Output Tokens	Cost (USD)
no-compression	46.00	87,325	1505	0.1867
remove-imports	39.00	82,997	1434	0.1775
merge-imports	45.00	86,034	1469	0.1838
remove-blank-lines	41.00	85,190	1628	0.1834
remove-comments	45.00	77,849	1457	0.1674
remove-docstrings	43.00	68,202	1384	0.1475
dedent	36.00	84,754	1662	0.1828
reduce-operators	41.00	80,132	1406	0.1715
short-vars	38.00	84,787	1590	0.1823
short-funcs	42.00	85,070	1547	0.1825
short-classes	45.00	83,423	1605	0.1797
short-vars-map	43.00	85,260	1549	0.1829
short-funcs-map	47.00	86,236	1623	0.1855
short-classes-map	46.00	86,398	1453	0.1844

Table 6.3: Single transformation ablation results on 100 SWE-bench Verified samples.

The results of the single transformation ablations are summarized in Table 6.3 and visualized in Figure 6.4. The most effective transformations in terms of token reductions are remove-comments and remove-docstrings, which decrease the average input size from 87,325 in the no-compression baseline to 77,849 and 68,202, respectively. These reductions translate directly into substantial per-instance cost savings, with remove-docstrings achieving the lowest overall cost at 0.1475 USD. The next most impactful transformation is reduce-operators, which lowers the average input tokens to 80,132 and cost to 0.1715 USD. Other transformations, such as shortening identifiers or merging imports, yield smaller but still measurable reductions.

In terms of performance, most transformations lead to minor decreases in resolution rate (typically below 5% relative to the baseline). The most pronounced degradation occurs with dedent, which reduces the resolution rate to 36%, corresponding to an 8% drop compared to no compression.

A more detailed cost breakdown is given in Figure 6.5. In contrast to the ranking step (where output cost dominated), the input cost constitutes the majority of overall cost. As expected, output cost remains roughly constant across transformations, while input cost varies depending on the applied transformation.

Renaming Identifiers

Figure 6.6 presents the results of the renaming ablation. We evaluate two variants of identifier renaming. In the first variant, all identifiers are trivially replaced with shortened names. In the second variant, we additionally provide the model with a context map linking each shortened identifier to its original name. To ensure that renaming reduces

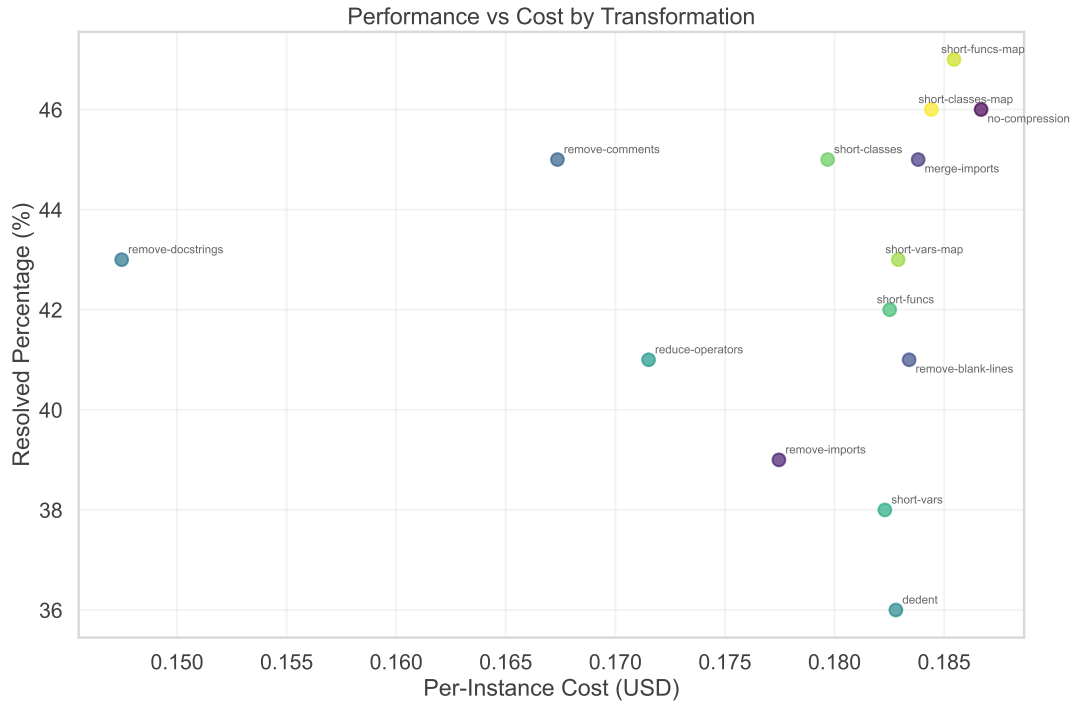

Figure 6.4: Performance vs. cost by transformation.

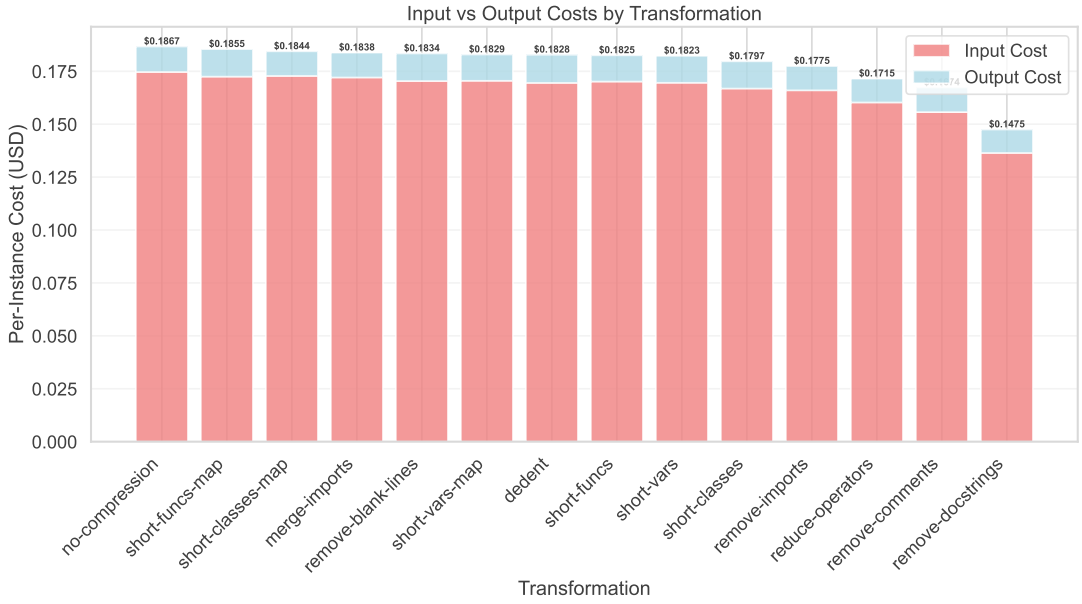

Figure 6.5: Input/output cost breakdown by transformation.

the total number of tokens, replacements are applied only when the token savings exceed the overhead of adding the corresponding mapping entry.

We compare both approaches by running the same transformation pipeline with and without including the context map in the prompt. In Figure 6.6, blue points represent the variant with the in-context map, and red points represent the variant without it. Overall, we find that providing the original identifier names is not necessary. The model performs slightly better without the context map, achieving an average resolution rate of 47.33% compared to 45.33% with the map included. The per-instance costs are nearly identical (0.1839 USD without the map and 0.1842 USD with it), with a minor increase aligning with the expected overhead of including the mapping.

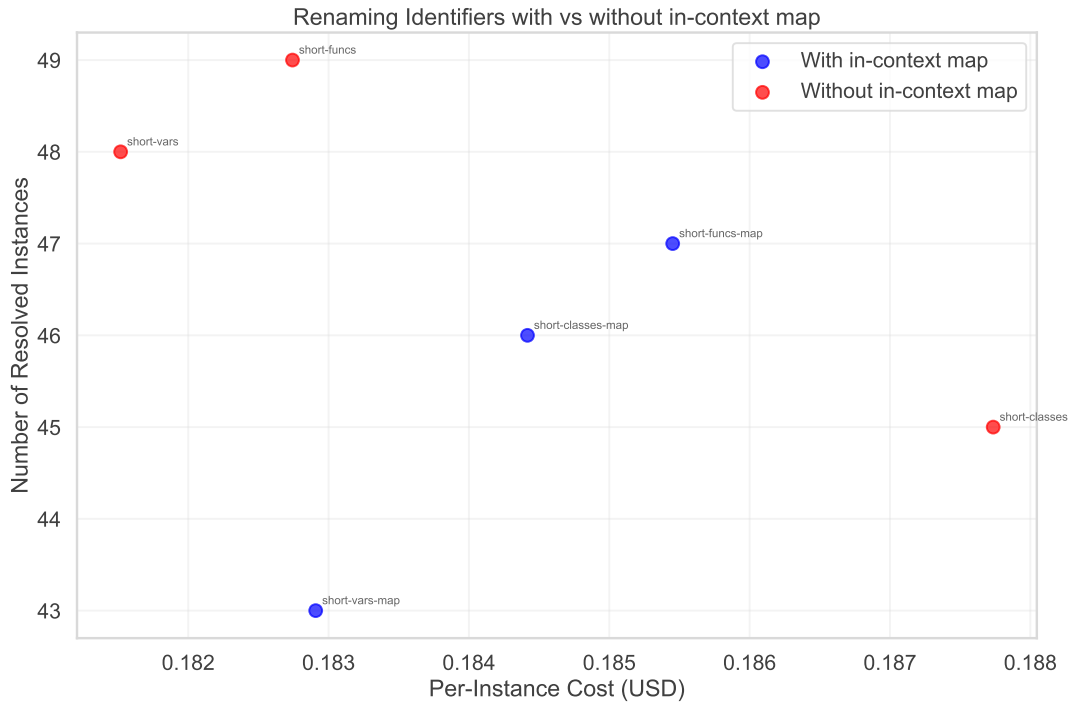

Figure 6.6: Renaming identifiers ablation study comparing variants with and without in-context maps. Results are shown in terms of per-instance cost and number of resolved instances.

Multiple Transformations

Figure 6.7 illustrates the effect of stacking multiple transformations. Transformations are applied in ascending order of their individual performance impact. Similar operations are grouped to reduce cost. For instance, the renaming identifiers step adds variable, function and class renaming. We also select remove-imports over merge-imports, as it has a higher potential for reducing token count.

The plot should be interpreted from left to right, where each point represents the cumulative application of all preceding transformations plus the one indicated by the label. We find that stacking transformations consistently reduces per-instance cost while maintaining performance close to the baseline. The primary exception is again the dedent transformation, which reduces the resolution rate to 27% compared to 47% in the no-compression baseline. Apart from this case, the stacked transformations demonstrate that substantial reductions in token usage can be achieved with only modest losses in resolution performance. Failure cases associated with the dedent transformation are analyzed in more detail later in this work.

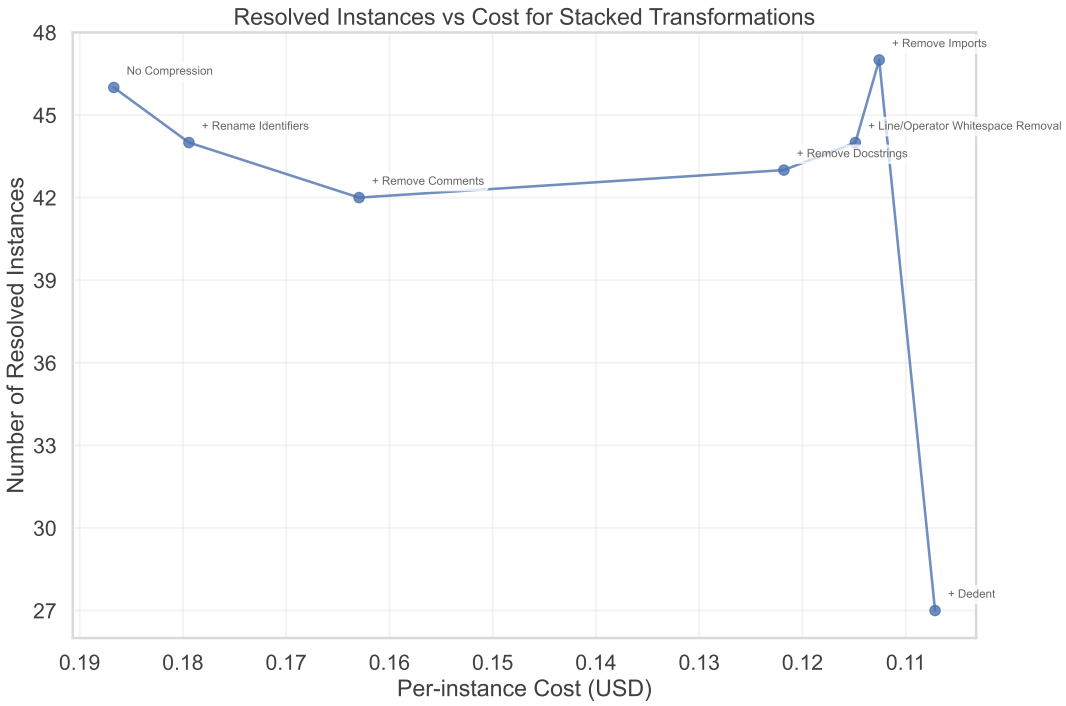

Figure 6.7: Resolved instances vs. cost when applying multiple transformations.

6.2.4 Robustness

In this section, we examine whether the observed token savings and associated performance effects generalize across models, repositories and issue difficulty levels.

Figure 6.8 compares GPT-4.1 and GPT-5-mini on a 100-sample subset using the same stacked transformation setup described earlier. The overall trend is consistent across both models. GPT-5-mini achieves slightly higher performance with fewer transformations but degrades more rapidly under stronger compression. It also exhibits fewer input tokens on average at equivalent transformation stages. Since these values reflect repair-stage inputs, the difference likely arises from variations in file selection during ranking, though

this aspect was not further analyzed. As before, dedent remains a clear outlier, causing a pronounced drop in performance.

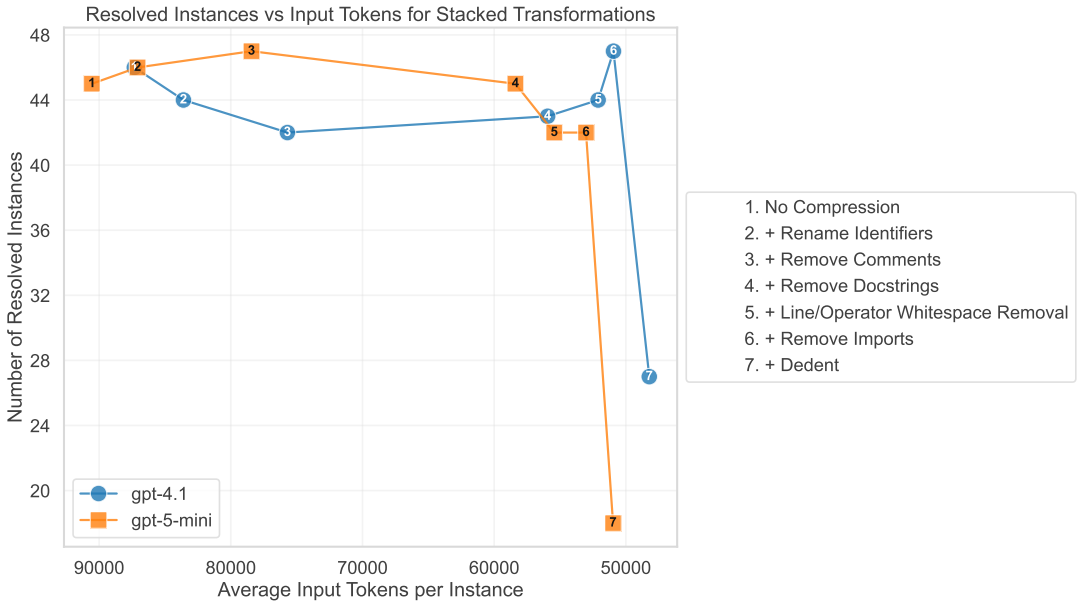

Figure 6.8: Resolved instances versus average input tokens per instance for stacked token-reduction transformations. Results are shown for GPT-4.1 and GPT-5-mini, with transformations applied incrementally in the order listed in the legend.

Figure 6.9 presents results grouped by issue difficulty. Difficulty levels correspond to the estimated time engineers reported for resolving each issue, divided into four categories. Both token reduction and performance effects remain stable across difficulty levels, indicating that compression effectiveness is largely independent of problem complexity.

Finally, Figure 6.10 reports the same analysis by repository. Of the 12 repositories in SWE-bench Verified, we exclude those with fewer than 10 samples, resulting in 10 repositories shown in the plot. We again observe consistent behavior across repositories. Both performance degradation and token savings remain stable, suggesting that the proposed compression methods generalize well across diverse codebases.

6.2.5 Failure Analysis

During evaluations with multiple source transformations (Figure 6.7 and Figure 6.8), most transformations caused at most a 4% performance decrease relative to the unmodified baseline. The only exception is the dedent transformation, which leads to a substantially larger drop in resolved instances. This section analyzes the cause of this degradation.

Figure 6.11 compares the instance-level outcomes for the baseline run and the run including the dedent transformation. Each instance along the x-axis corresponds to a

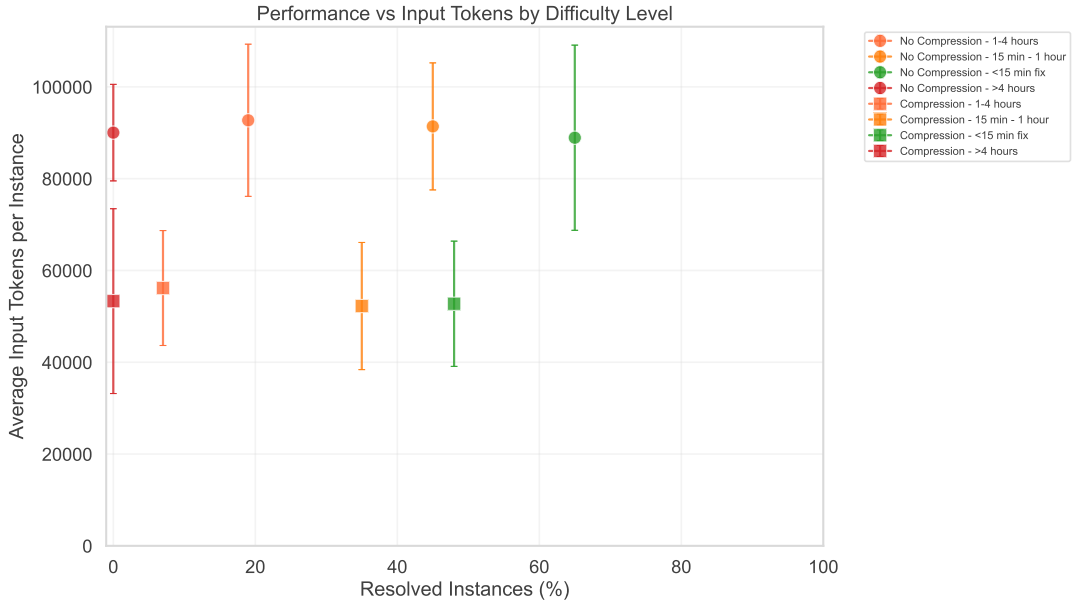

Figure 6.9: Performance vs. Input Tokens by Difficulty Level

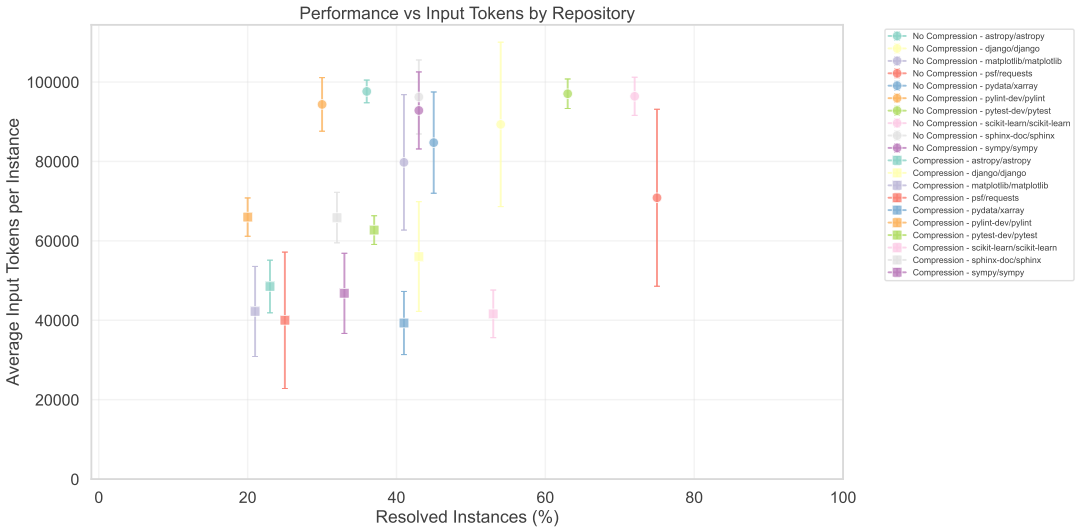

Figure 6.10: Performance vs. Input Tokens by Repository

benchmark problem, with resolved cases shown in green and unresolved cases in red. The plot reveals that all instances resolved under dedent are a strict subset of those resolved in the baseline. Thus, the degradation likely cannot be attributed to stochastic variation but rather to transformation-induced failures. Additional plots illustrating this pattern are provided in Appendix B.

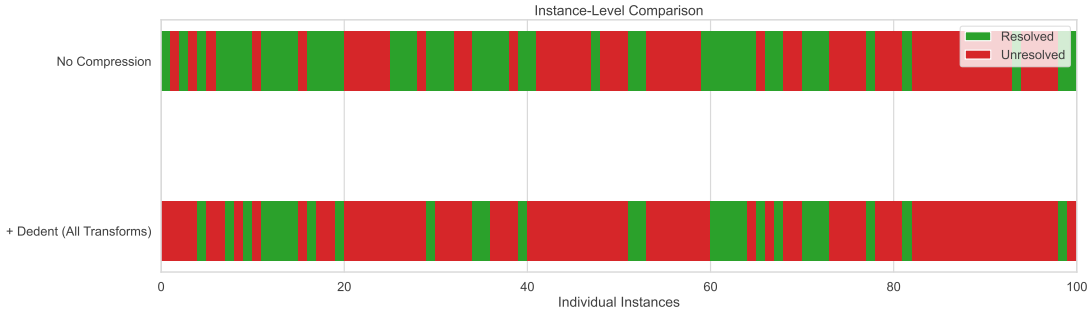

Figure 6.11: Comparing instance-level resolutions when no transformations (top) and all transformations (bottom) are used.

To further investigate, we examine individual failure cases. Evaluation logs indicate that for several instances, the applied patches are syntactically invalid after dedentation. The model occasionally fails to restore the required four-space indentation in its patch outputs, violating the instructed format. This frequently produces syntactically invalid Python code that results in universal test failures.

A representative example is `django-12155`. The corresponding patch is applied cleanly, but results in a malformed function body:

```
diff --git a/django/contrib/admindocs/utils.py b/django/contrib/admindocs/
    utils.py
index 4c0e7e2a56..2c5a41aae9 100644
--- a/django/contrib/admindocs/utils.py
+++ b/django/contrib/admindocs/utils.py
@@ -33,9 +33,9 @@ def trim_docstring(docstring):
     if not docstring or not docstring.strip():
         return ''
     # Convert tabs to spaces and split into lines
-    lines = docstring.expandtabs().splitlines()
-    indent = min(len(line) - len(line.lstrip()) for line in lines if line.
    lstrip())
-    trimmed = [lines[0].lstrip()] + [line[indent:].rstrip() for line in
    lines[1:]]
+    lines=docstring.expandtabs().splitlines()
+ indent=min(len(line)-len(line.lstrip())for line in lines[1:] if line.lstrip
    ())
+ trimmed=[lines[0].lstrip()]+[line[indent:].rstrip()for line in lines[1:]]
     return "\n".join(trimmed).strip()
```

Lines originally nested inside a function lost their indentation hierarchy, producing the following Python error:

```
IndentationError: unindent does not match any outer indentation level
```

These observations indicate that the degradation is primarily caused by syntactic failures rather than semantic information loss. Correcting this issue (e.g., by normalizing indentation in the model output prior to patch application) would likely restore dedent performance to levels comparable with other transformations.

CHAPTER 7

Conclusion

LLM-based SWE agents must reason over large codebases, making source code tokens a primary bottleneck in context length. This thesis proposes the use of code minification as a means to decrease token consumption in LLM-based agents. The results demonstrate that minification transformations can substantially reduce context size, improving the cost-effectiveness of repair agents while maintaining competitive performance.

7.1 Research Questions

RQ1: How are tokens allocated in LLM-based software-engineering agents? We conduct a quantitative analysis of token distribution across workflow stages and content types. Results show that code tokens dominate total context consumption in our setup, accounting for more than 90% of all processed tokens. Within code, non-functional lexical elements such as comments, docstrings and whitespace contribute a substantial fraction of the total size. These findings motivate a focus on code-centric transformations to reduce token load.

RQ2: Which token-reducing input transformation strategies are suitable to effectively address observed token bottlenecks? Building on the insights from RQ1, we implement a set of semantics-preserving minification transformations, including comment and docstring removal, whitespace reduction and identifier shortening. These transformations are integrated into the agent pipeline to prepare for empirical evaluation.

RQ3: What is the impact of implemented input transformation strategies on token consumption and task performance? Empirical evaluation on SWE-bench Verified shows that code minification reduces average input token usage by approximately 42%, while decreasing the resolution rate by only 12%. This demonstrates that significant efficiency gains can be achieved with minimal loss in task performance.

RQ4: How robust and generalizable are observed savings and trade-offs? We evaluate the consistency of token savings and performance across models, repositories and issue difficulty. The results exhibit stable trends across all settings. GPT-4.1 and GPT-5-mini show similar behavior, with dedent as the primary outlier, causing a pronounced performance drop. Token reductions and performance effects also remain consistent across difficulty levels and repositories, indicating that the proposed transformations generalize well across different conditions.

7.2 Limitations & Future Work

While the results demonstrate the effectiveness of token-reduction transformations for code-intensive repair tasks, several limitations constrain the precision and generality of the findings. The following section discusses these limitations and outlines directions for future research.

7.2.1 Stochastic Variability in Model Behavior

A primary limitation of this study arises from the stochastic nature of large language models. Even under identical configurations, we observe variations in resolution rates across repair runs. Furthermore, the set of files selected during repair differs between runs, directly affecting the measured input and output token counts. Consequently, exact numerical results should be interpreted with caution, particularly when differences are small. Nonetheless, we argue that the general trends observed in our experiments remain valid and provide meaningful evidence of the effectiveness of the proposed transformations.

Future work could mitigate these sources of variability through several strategies. One approach is to fix the file selection process during repair, allowing for a more controlled assessment of the cost impact of token-reduction transformations. Another is to perform multiple runs per configuration and report averaged results to improve measurement stability.

7.2.2 Agent-Specific Design Bias

Another limitation of this study is that all experiments were conducted using a single agent that is particularly heavy on source code input. Consequently, the observed effects may not generalize to systems in which code ingestion constitutes a smaller proportion of the total context. Nonetheless, we hypothesize that source code processing represents a common bottleneck across a broad range of software engineering agents. When addressing issues, agents typically must read and interpret at least the directly relevant portions of the codebase to accurately localize and reason about the problem, which is an inherently challenging and context-intensive process.

7.2.3 Validity of Generated Patches

Recent work [85] has shown that automatically generated patches may produce behavior that satisfies test cases but diverges from developer intent. Specifically, LLM-based repair agents can generate plausible fixes that pass all tests while failing to resolve the root cause of the underlying issue or introducing unintended side effects. This study does not assess the semantic validity of generated patches beyond their test outcomes. Consequently, some patches classified as successful may not represent truly correct repairs. Future work could incorporate additional validation steps such as human evaluation, to assess the quality and correctness of the produced repairs.

7.2.4 Formatting Artifacts in Generated Diffs

Another limitation of the current implementation concerns the handling of code formatting during the repair process. In our setup, the LLM operates on minified code during the repair stage, producing minified SEARCH and REPLACE blocks that define the edits. The minified `SEARCH` block is then matched against the original (unminified) code using a search procedure that is agnostic to formatting changes introduced by minification. The corresponding `REPLACE` block is subsequently applied to the matched region in the original source. However, the transformations within the `REPLACE` block itself are not reversed, which can result in diffs containing unintended formatting modifications.

An example of such an artifact is shown in Listing 7.1, where spacing around operators is removed as a side effect of minification (in addition to the intended import addition):

```
-from .util import _str_to_num, _is_int, translate, _words_group
+from.util import _str_to_num,_is_int,translate,_words_group,
    decode_ascii
```

Listing 7.1: Formatting artifacts introduced during patch application.

This issue can be addressed with additional engineering effort. For instance, by applying a formatter that enforces the original code's style rules or by maintaining source maps that allow reversal of the applied transformations. We currently implement such reversibility only for the renaming transformations, as identifier renaming would otherwise break the test suite.

APPENDIX A

Additional Instructions

This appendix lists additional instructions injected into the LLM prompts for specific transformations and repository representations.

A.1 Repository Representation Instructions

A.1.1 Int-Folder Representation

```
Below you can find the full list of file paths for the repository.
If a path name is an int, you can look up the actual path in the mapping above.

For instance, if the listed name is 1/def.py and there is an entry 1:abc,
then the actual full path would be abc/def.py.
```

A.1.2 Int-Path Representation

```
Below you can find the full list of file paths for the repository.
If a folder name is an int, you can look up the actual name in the
folder mapping above.

For instance, if the listed name is 1/2 and there are entries 1:abc and 2:def.py,
then the actual full path would be abc/def.py.
```

A.2 Transformation Instructions

A.2.1 Renaming Transformations (with Source Maps)

```
SOURCE MAPS (for shortened identifiers):

The following mappings show the relationship between shortened identifiers
```

```
and their original names:

<transformation_name>:
<shortened> -> <original>

Use the original names when you output the search and replace blocks.
For instance, if the source maps contain an entry '- a -> b',
then a is the shortened name that you will find in the code
and b is the original. In this case, use b instead of a when
you output the search and replace block.
```

A.2.2 Dedent Transformation

```
Always output the SEARCH and REPLACE block with a four-space
indentation style, even if you receive source files that
use less indentation.
```

APPENDIX B

Instance-Level Resolutions

This appendix provides a detailed instance-level analysis of resolution outcomes across different transformation configurations. The figures below illustrate which individual SWE-bench instances were successfully resolved under each cumulative setup (Figure B.1) and compare the results between the baseline (no transformations) and the full transformation configuration (Figure B.2). These visualizations highlight the degree of overlap and correlation in successful instances across different setups.

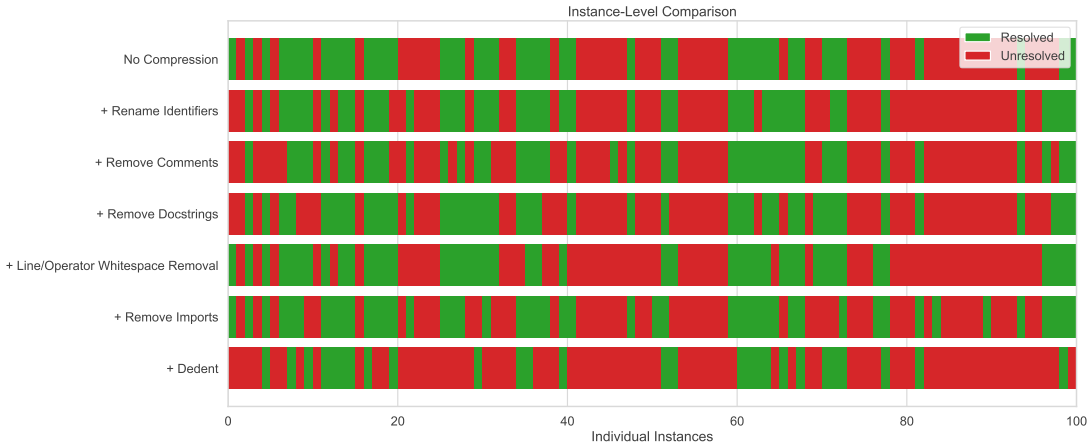

Figure B.1: Instance-level comparison across incremental transforms. Each horizontal row corresponds to a cumulative transform configuration. Green indicates resolved instances and red indicates unresolved instances.

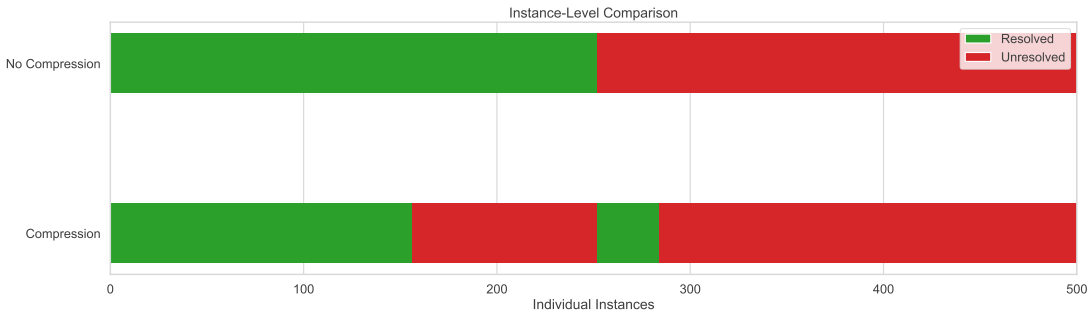

Figure B.2: Instance-level comparison across full runs (with no transformations and with all transformations except dedent applied). Instances are sorted such that they respect the following ordering: 1. instances resolved if no transformations were applied; 2. instances resolved only with all transformations; 3. remaining instances.

Overview of Generative AI Tools Used

Overall ChatGPT-5 and Cursor were employed for supporting tasks and coding assistance that were not central to the core contributions of this thesis.

Writing ChatGPT was used to refine phrasing, suggesting alternative formulations and synonyms. It also assisted in reviewing the final text for grammatical accuracy and spelling corrections.

Code ChatGPT and Cursor were used to generate plotting scripts and small code snippets based on detailed prompts.

Acknowledgements, Abstract and Overview of Generative AI Tools The English texts for the *Acknowledgements*, *Abstract*, and *Overview of Generative AI Tools* sections were translated into German using ChatGPT. Manually post-edited versions of these translations were incorporated as *Danksagung*, *Kurzfassung*, and *Übersicht verwendeter Hilfsmittel*.

Introduction ChatGPT was used to generate the LaTeX code for the mathematical problem formulation.

Background and Related Work ChatGPT served as a complementary research tool to identify relevant literature alongside manual research.

Evaluation LaTeX result tables were generated with ChatGPT based on the experimental raw data.

Übersicht verwendeter Hilfsmittel

Overall ChatGPT-5 und Cursor wurden für unterstützende Aufgaben eingesetzt, die nicht zum zentralen Beitrag dieser Arbeit gehörten.

Writing ChatGPT wurde zur Textüberarbeitung eingesetzt, um alternative Formulierungen und Synonyme vorzuschlagen. Zudem unterstützte es bei der abschließenden Überprüfung des Textes auf grammatikalische Richtigkeit und Rechtschreibung.

Code ChatGPT und Cursor wurden verwendet, um Skripte zur Erstellung von Plots und kleiner Codestücke anhand detaillierter Anweisungen zu generieren.

Acknowledgements, Abstract and Overview of Generative AI Tools Die englischen Texte der Abschnitte *Acknowledgements*, *Abstract* und *Overview of Generative AI Tools* wurden mit ChatGPT ins Deutsche übersetzt. Manuell leicht überarbeitete Versionen dieser Übersetzungen wurden als *Danksagung*, *Kurzfassung* und *Übersicht verwendeter Hilfsmittel* übernommen.

Introduction ChatGPT wurde zur Erstellung des LaTeX-Codes für die mathematische Problemformulierung verwendet.

Background and Related Work ChatGPT diente als ergänzendes Recherchewerkzeug zur Identifizierung relevanter Literatur.

Evaluation LaTeX-Tabellen für die Ergebnisse wurden von ChatGPT auf Grundlage der bereitgestellten Rohdaten erstellt.

List of Figures

List of Tables

List of Algorithms

Bibliography

[1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[2] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts llm performance. Technical report, Chroma, July 2025.

[3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Trans. Assoc. Comput. Linguistics*, 12:157–173, 2024.

[4] Stan Franklin and Arthur C. Graesser. Is it an agent, or just a program?: A taxonomy for autonomous agents. In Jörg P. Müller, Michael J. Wooldridge, and Nicholas R. Jennings, editors, *Intelligent Agents III, Agent Theories, Architectures, and Languages, ECAI '96 Workshop (ATAL), Budapest, Hungary, August 12-13, 1996, Proceedings*, volume 1193 of *Lecture Notes in Computer Science*, pages 21–35. Springer, 1996.

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nat.*, 518(7540):529–533, 2015.

[6] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[8] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic

actor. In Jennifer G. Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 1856–1865. PMLR, 2018.

[9] Brenden M. Lake, Tomer D. Ullman, Joshua B. Tenenbaum, and Samuel J. Gershman. Building machines that learn and think like people. *CoRR*, abs/1604.00289, 2016.

[10] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[12] OpenAI and Josh Achiam et al. Gpt-4 technical report, 2024.

[13] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023.

[14] Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023.

[15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

[17] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving AI tasks with chatgpt and its friends in hugging face. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[18] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers Comput. Sci.*, 18(6):186345, 2024.

[19] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 15174–15186. Association for Computational Linguistics, 2024.

[20] Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In Sean Follmer, Jeff Han, Jürgen Steimle, and Nathalie Henry Riche, editors, *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023, San Francisco, CA, USA, 29 October 2023- 1 November 2023*, pages 2:1–2:22. ACM, 2023.

[21] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.

[22] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: program-aided language models. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR, 2023.

[23] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

[24] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[25] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, pages 17682–17690. AAAI Press, 2024.

[26] Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[27] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: empowering large language models with optimal planning proficiency. *CoRR*, abs/2304.11477, 2023.

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[29] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified, 2024. Accessed: 2025-10-04.

[30] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In Maria Christakis and Michael Pradel, editors, *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software*

Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024, pages 1592–1604. ACM, 2024.

[31] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *CoRR*, abs/2407.01489, 2024.

[32] Mingjian Jiang, Yangjun Ruan, Luis A. Lastras, Pavan Kapanipathi, and Tatsunori Hashimoto. Putting it all into context: Simplifying agents with lclms. *CoRR*, abs/2505.08120, 2025.

[33] Lizhou Fan, Wenyue Hua, Lingyao Li, Haoyang Ling, and Yongfeng Zhang. Nphardeval: Dynamic benchmark on reasoning ability of large language models via complexity classes. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 4092–4114. Association for Computational Linguistics, 2024.

[34] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 3102–3116. Association for Computational Linguistics, 2023.

[35] Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, and Jian Zhang. On the tool manipulation capability of open-source large language models. *CoRR*, abs/2305.16504, 2023.

[36] Karthik Valmeekam, Matthew Marquez, Alberto Olmo Hernandez, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[37] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[38] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents.

In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[39] Mark Chen et al. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.

[40] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *CoRR*, abs/1508.07909, 2015.

[41] Philip Gage. A new algorithm for data compression. *C Users J.*, 12(2):23–38, February 1994.

[42] Machel Reid et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *CoRR*, abs/2403.05530, 2024.

[43] Greg Kamradt. Needle In A Haystack – Pressure Testing LLMs. `https://github.com/gkamradt/LLMTest_NeedleInAHaystack`, 2023. GitHub repository.

[44] Krithik Vishwanath, Anton Alyakin, Daniel Alexander Alber, Jin Vivian Lee, Douglas Kondziolka, and Eric Karl Oermann. Medical large language models are easily distracted, 2025.

[45] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? A large-scale open domain question answering dataset from medical exams. *CoRR*, abs/2009.13081, 2020.

[46] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227. PMLR, 2023.

[47] Meghana Arakkal Rajeev, Rajkumar Ramamurthy, Prapti Trivedi, Vikas Yadav, Oluwanifemi Bamgbose, Sathwik Tejaswi Madhusudhan, James Zou, and Nazneen Rajani. Cats confuse reasoning LLM: query agnostic adversarial triggers for reasoning models. *CoRR*, abs/2503.01781, 2025.

[48] Gheorghe Comanici et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *CoRR*, abs/2507.06261, 2025.

[49] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. Llms get lost in multi-turn conversation. *CoRR*, abs/2505.06120, 2025.

[50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle

Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017.

[51] MDN. Minification — mdn web docs. Accessed: 2025-10-16.

[52] Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. Prompt compression for large language models: A survey. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, pages 7182–7195. Association for Computational Linguistics, 2025.

[53] Jesse Mu, Xiang Li, and Noah D. Goodman. Learning to compress prompts with gist tokens. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[54] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 3829–3846. Association for Computational Linguistics, 2023.

[55] David Wingate, Mohammad Shoeybi, and Taylor Sorensen. Prompt compression and contrastive conditioning for controllability and toxicity reduction in language models. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 5621–5634. Association for Computational Linguistics, 2022.

[56] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[57] Zongqian Li, Yixuan Su, and Nigel Collier. 500xcompressor: Generalized prompt compression for large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 25081–25091. Association for Computational Linguistics, 2025.

[58] Jun Gao, Qi Lv, Zili Wang, Tianxiang Wu, Ziqiang Cao, and Wenjie Li. Uniicl: An efficient unified framework unifying compression, selection, and generation, 2025.

[59] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 6342–6353. Association for Computational Linguistics, 2023.

[60] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

[61] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 13358–13376. Association for Computational Linguistics, 2023.

[62] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 1658–1677. Association for Computational Linguistics, 2024.

[63] Hoyoun Jung and Kyung-Joong Kim. Discrete prompt compression with reinforcement learning. *IEEE Access*, 12:72578–72587, 2024.

[64] Qingyue Wang, Yanhe Fu, Yanan Cao, Shuai Wang, Zhiliang Tian, and Liang Ding. Recursively summarizing enables long-term dialogue memory in large language models. *Neurocomputing*, 639:130193, 2025.

[65] Yu-Neng Chuang, Tianwei Xing, Chia-Yuan Chang, Zirui Liu, Xun Chen, and Xia Ben Hu. Learning to compress prompt in natural language formats. In Kevin Duh, Helena Gómez-Adorno, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 7756–7767. Association for Computational Linguistics, 2024.

[66] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: improving retrieval-augmented lms with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[67] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and

Dongmei Zhang. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, pages 963–981. Association for Computational Linguistics, 2024.

[68] Pengfei He, Shaowei Wang, and Tse-Hsun Chen. CODEPROMPTZIP: code-specific prompt compression for retrieval-augmented generation in coding tasks with lms. *CoRR*, abs/2502.14925, 2025.

[69] Guang Yang, Yu Zhou, Wei Cheng, Xiangyu Zhang, Xiang Chen, Terry Yue Zhuo, Ke Liu, Xin Zhou, David Lo, and Taolue Chen. Less is more: Docstring compression in code generation. *CoRR*, abs/2410.22793, 2024.

[70] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[71] Tiantian Gan and Qiyao Sun. RAG-MCP: mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *CoRR*, abs/2505.03275, 2025.

[72] Xiaonan Li and Xipeng Qiu. Mot: Memory-of-thought enables chatgpt to self-improve. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 6354–6374. Association for Computational Linguistics, 2023.

[73] Anthropic Engineering. How we built our multi-agent research system. Anthropic blog, June 2025. Published June 13, 2025.

[74] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-one: A generalist multi-agent system for solving complex tasks. *CoRR*, abs/2411.04468, 2024.

[75] Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing wikipedia-like articles from scratch with large language models. In Kevin Duh, Helena Gómez-Adorno, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 6252–6278. Association for Computational Linguistics, 2024.

[76] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: learning to synergize memory and reasoning for efficient long-horizon agents. *CoRR*, abs/2506.15841, 2025.

[77] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 2369–2380. Association for Computational Linguistics, 2018.

[78] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.

[79] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. *CoRR*, abs/2508.21433, 2025.

[80] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. Improving the efficiency of LLM agent systems through trajectory reduction. *CoRR*, abs/2509.23586, 2025.

[81] Ryan Ehrlich, Bradley C. A. Brown, Jordan Juravsky, Ronald Clark, Christopher Ré, and Azalia Mirhoseini. Codemonkeys: Scaling test-time compute for software engineering. *CoRR*, abs/2501.14723, 2025.

[82] Anthropic. Claude 3.5 sonnet. https://www.anthropic.com/news/claude-3-5-sonne, June 21 2024.

[83] dzhuang. pyminifier3. https://github.com/dzhuang/pyminifier3.

[84] OpenAI. Introducing gpt-5. `https://openai.com`, August 2025. Accessed: 2025-09-11.

[85] You Wang, Michael Pradel, and Zhongxin Liu. Are "solved issues" in swe-bench really solved correctly? an empirical study. *CoRR*, abs/2503.15223, 2025.